



EEE 121

Data Structures and Algorithms for EEE



Minimum Spanning Tree

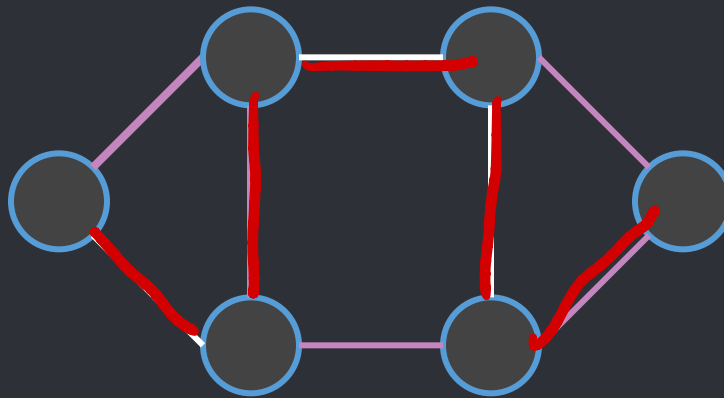
Prim's Algorithm, Kruskal's Algorithm

• Spanning Tree

A spanning tree of a graph G is a subgraph T that is:

- connected
 - acyclic
 - includes all vertices of G - spanning
- } tree

Example:

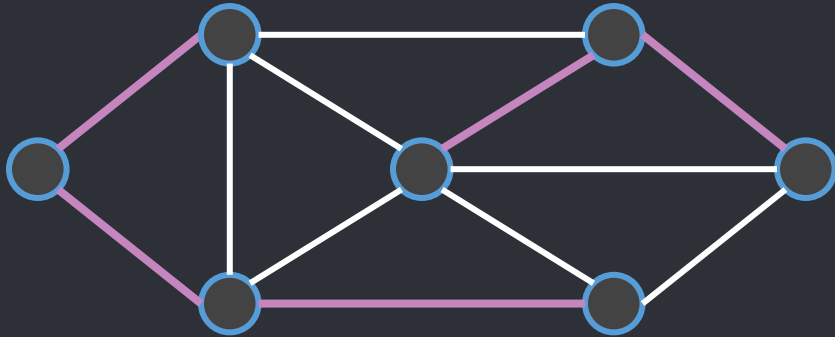


no of vertices

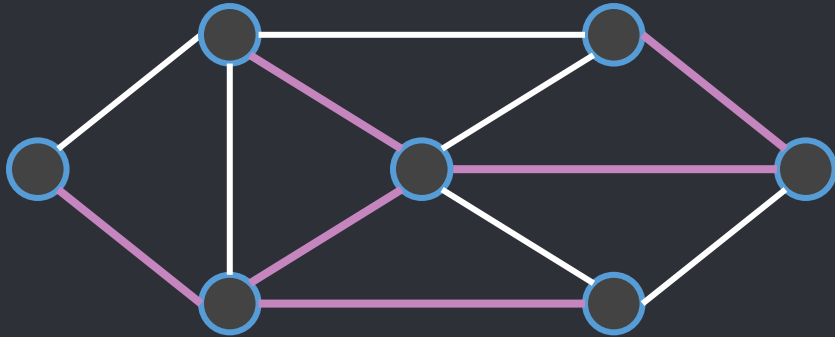
Note: The number of edges in a spanning tree is $|V| - 1$.

- Spanning Tree

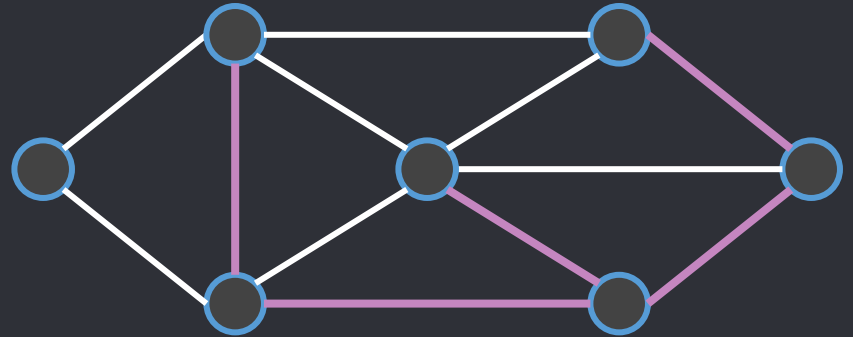
Not a ST - disconnected



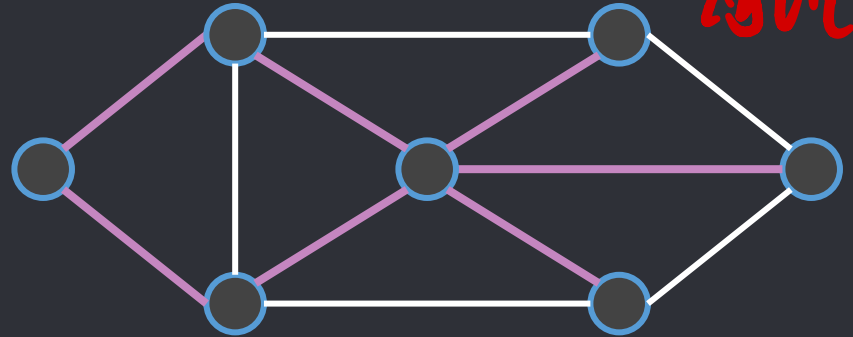
ST - valid!



Not a ST - not spanning

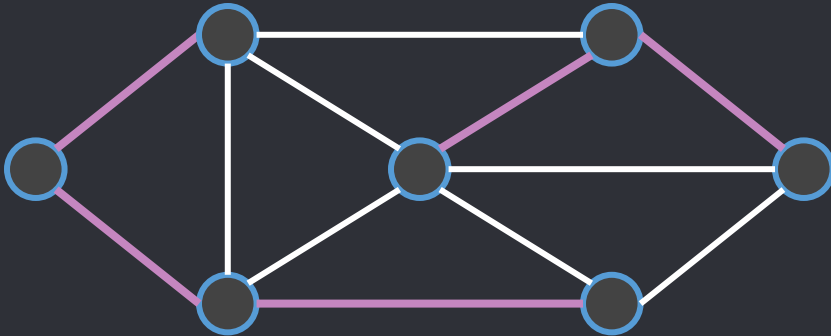


Not a ST - there is a cycle

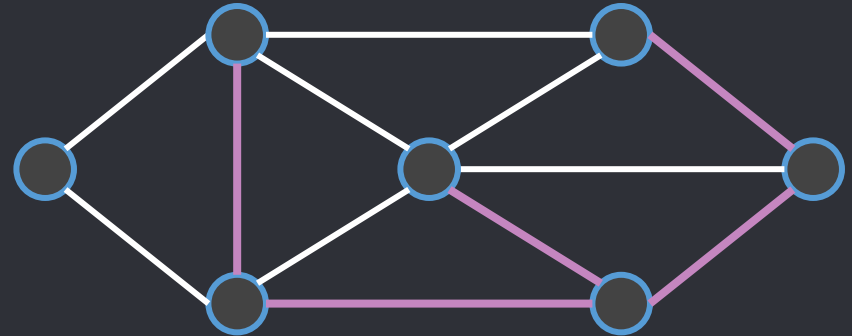


- Spanning Tree

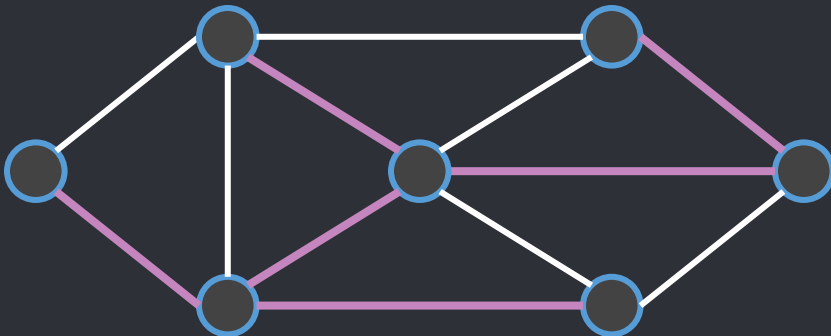
not connected



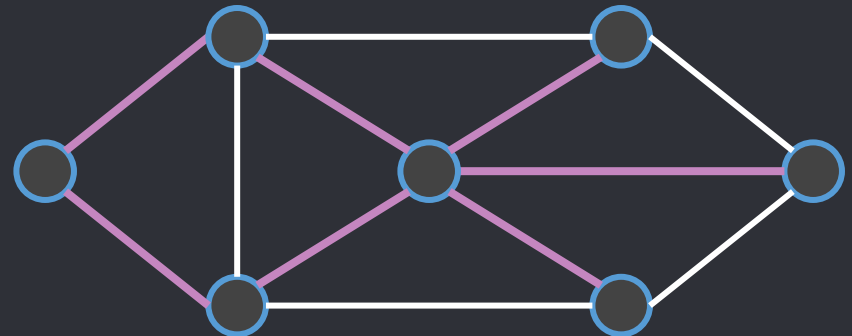
not spanning



spanning tree



not acyclic



• Minimum Spanning Tree (MST)

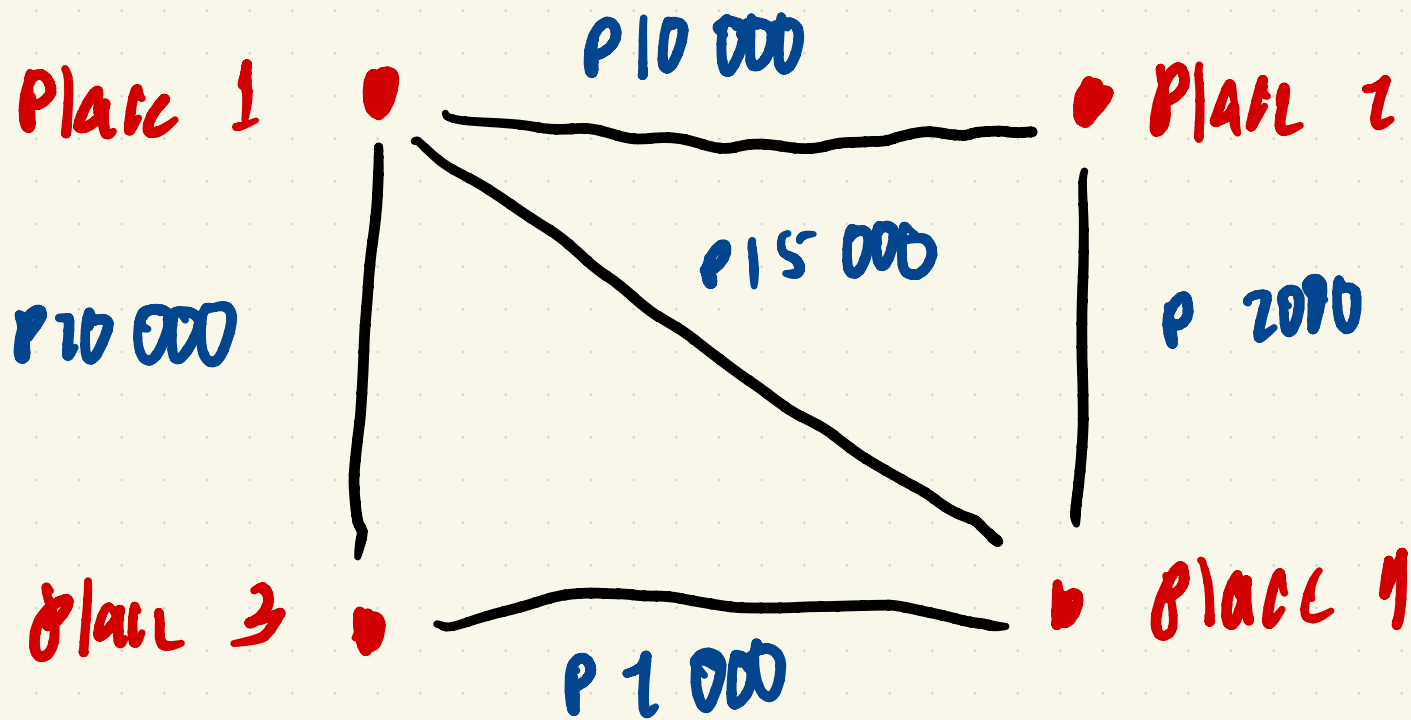
A minimum spanning tree is a spanning tree of a graph with minimum total weight.

Applications:

- Distribution networks
 - telecommunication
 - electricity
 - water
- Transportation networks
 - routes

BFS - are all nodes connected!
Dijkstra's - what is the shortest path from node k to others

MST - how can I connect all of my nodes with the least cost



Goal: connect all places

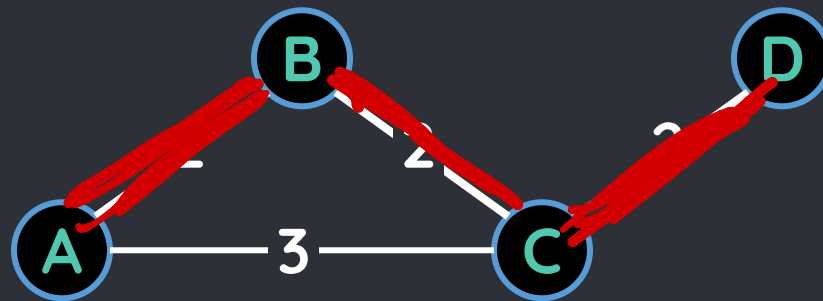
$$\times (1,2) + (1,3) + (1,4) = 10000 + 20000 + 15000 = 45000$$

$$\checkmark (1,2) + (1,4) + (4,3) = 10000 + 1000 + 2000 = 14000$$

↳ a lot cheaper

• MST

Find the MST for the graph below.



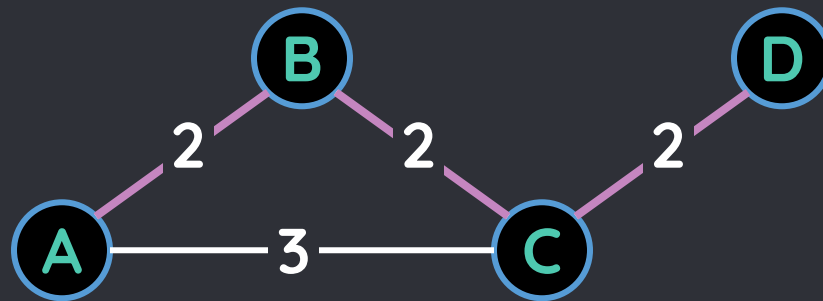
$$(A, C) + (C, B) + (C, D) = 3 + 2 + 2 = 7$$

$$(A, B) + (A, C) + (C, D) = 1 + 3 + 2 = 6$$

$$(A, B) + (B, C) + (C, D) = 1 + 2 + 2 = 5$$

- **MST**

Find the MST for the graph below.



• MST Simplifying Assumptions

○ Some assumptions when finding an MST

- Graph is undirected
 - standard MST problem
- Graph is weighted
 - solution is non-trivial
- Graph is connected
 - MST exists
- Edge weights are distinct
 - MST is unique

• Finding the MST: Naive Way

One way to find the MST is to generate all the possible spanning trees; then, get the spanning tree with the minimum total weight.

Problem:

- Time complexity is too high, $O(2^{|E|})$

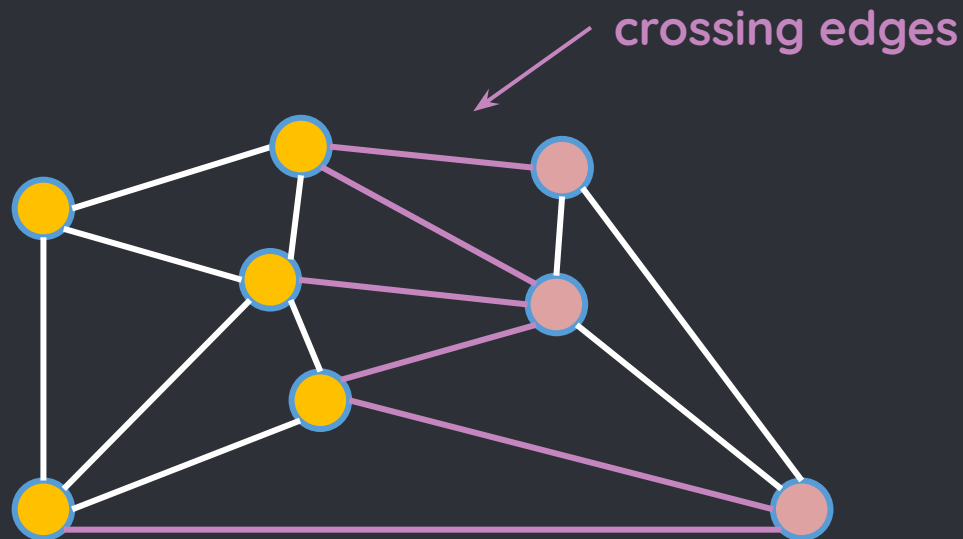
Goal:

- Determine a way to find the MST without generating (all) combinations of the graph.

• Finding the MST: Graph Cut

A (graph) cut is an assignment of a graph's nodes to two non-empty sets (or subgraphs).

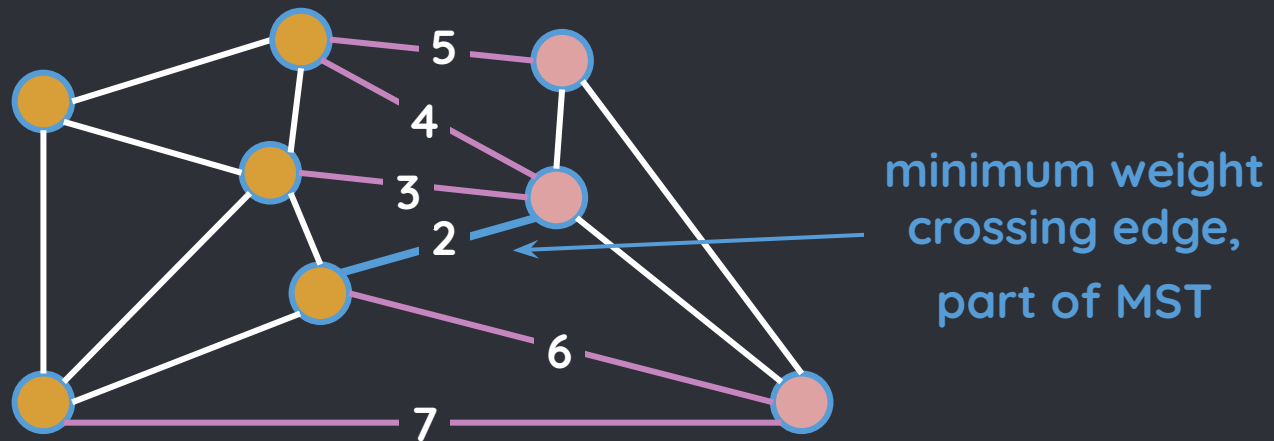
A crossing edge is an edge which connects the two sets of a (graph) cut.



• Finding the MST: Cut Property

Cut property: For any given graph cut, the minimum weight crossing edge is in the MST.

Cut optimality: Every edge in the MST is a result of the cut property.



Single edge is the min crossing edge, it belongs to the MST.

• Cut Property Proof

Proof by Contradiction

↑ This is true

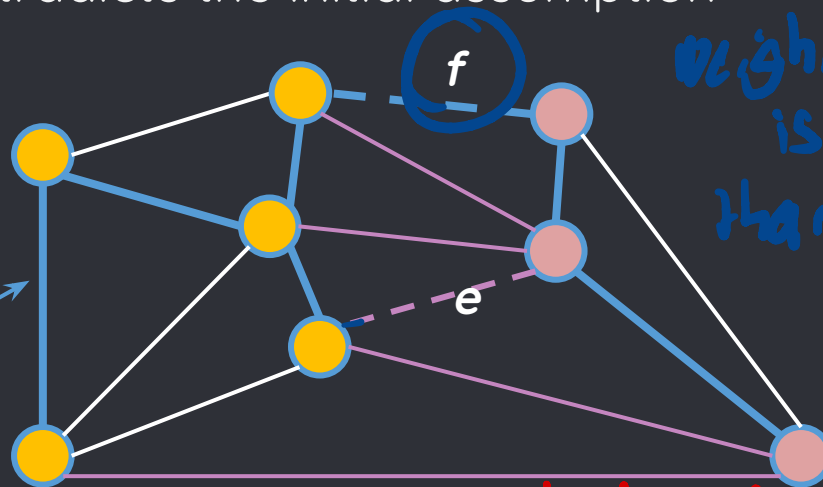
Suppose that the minimum crossing edge e were not in the **MST**.

- Adding e to the MST creates a cycle
- Some other edge f in the MST must also be a crossing edge
- Removing f and adding e is a lower weight spanning tree which contradicts the initial assumption

contradicts



MST does not contain e

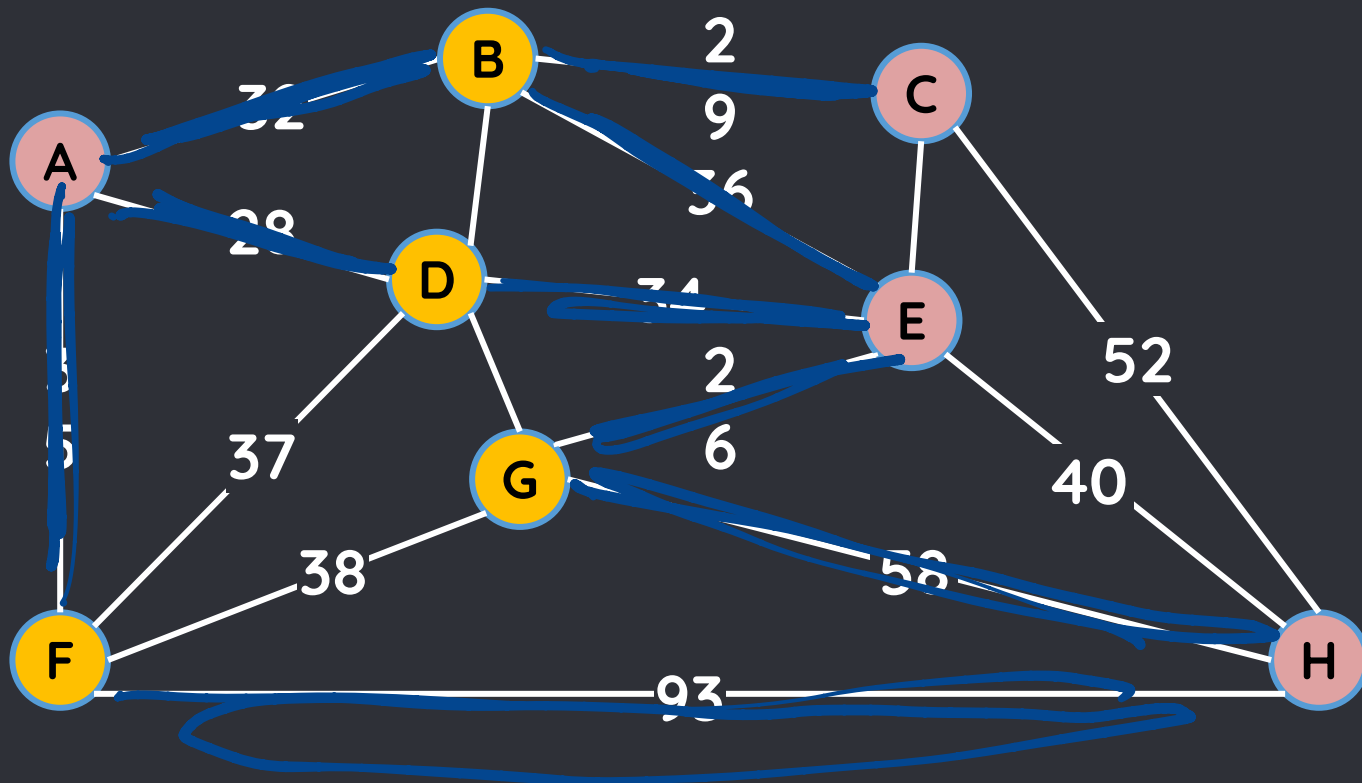


weight of f is larger than e

weight of MST with f > weight of MST with e

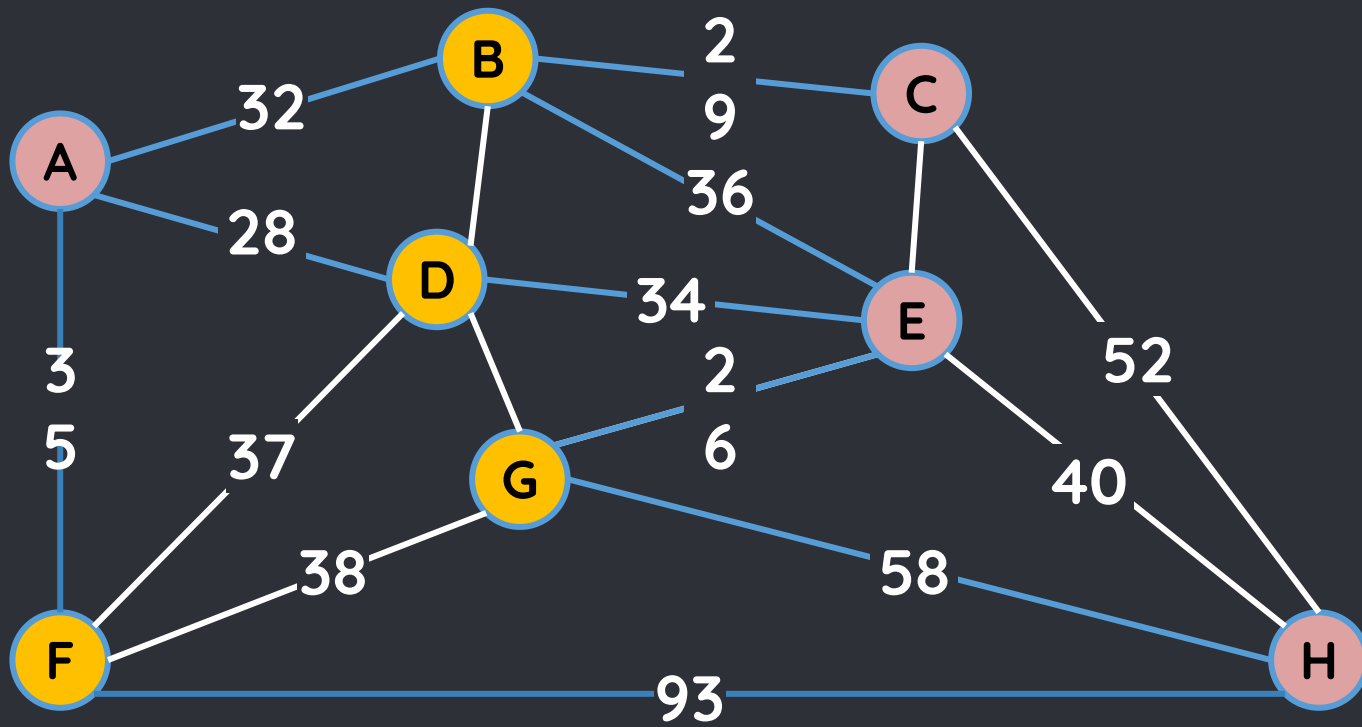
• Cut Property Example

Which edge is the minimum weight edge crossing the cut $\{A, C, E, H\}$?



• Cut Property Example

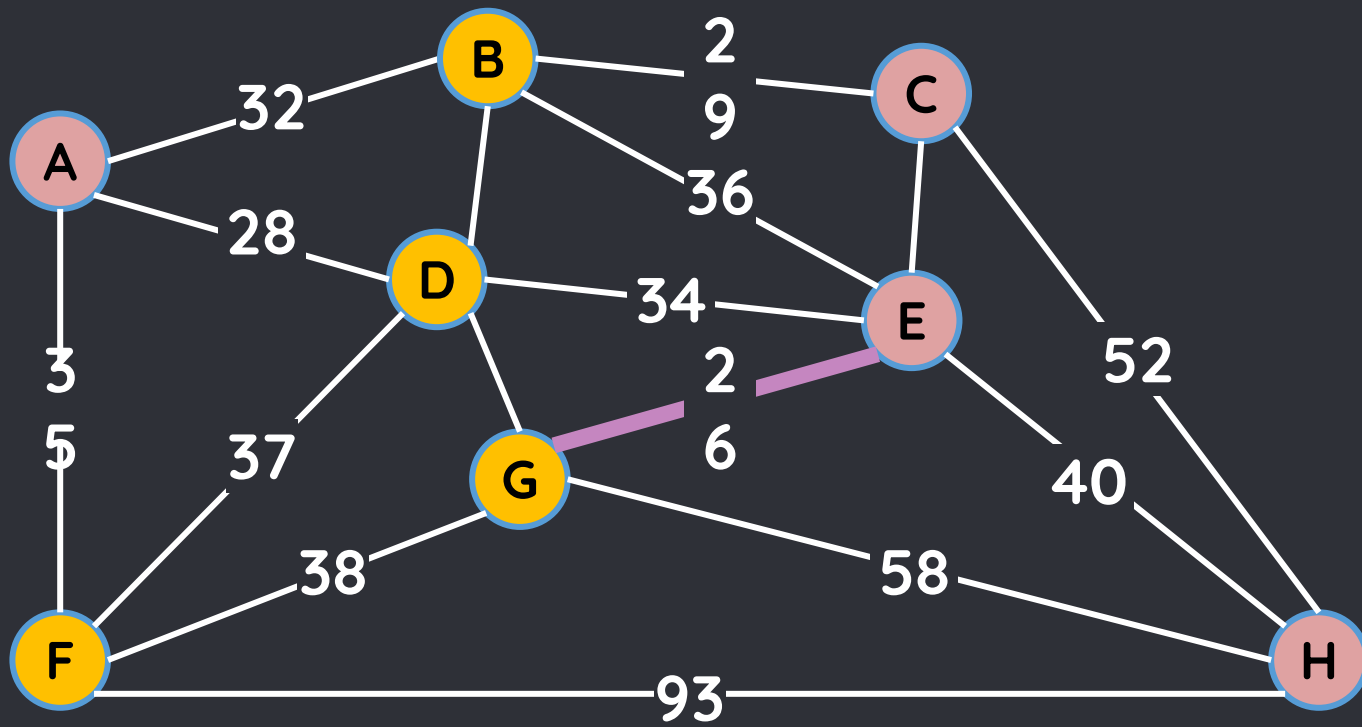
Which edge is the minimum weight edge crossing the cut $\{A, C, E, H\}$?



These are the crossing edges

• Cut Property Example

Which edge is the minimum weight edge crossing the cut $\{A, C, E, H\}$?



(G, E) must be part of MST

• Generic MST Algorithm

Start with no edges in the MST

Until there are $|V| - 1$ edges:

- Find a graph cut that has no crossing edges in the MST (yet)
- Add the smallest crossing edge to the MST

Problem:

- How do we systematically generate cuts without going through all combinations?
- Do we immediately find cuts which have no crossing edge?

• Prim's Algorithm - Concept

○ Select any arbitrary starting node and include in the MST.

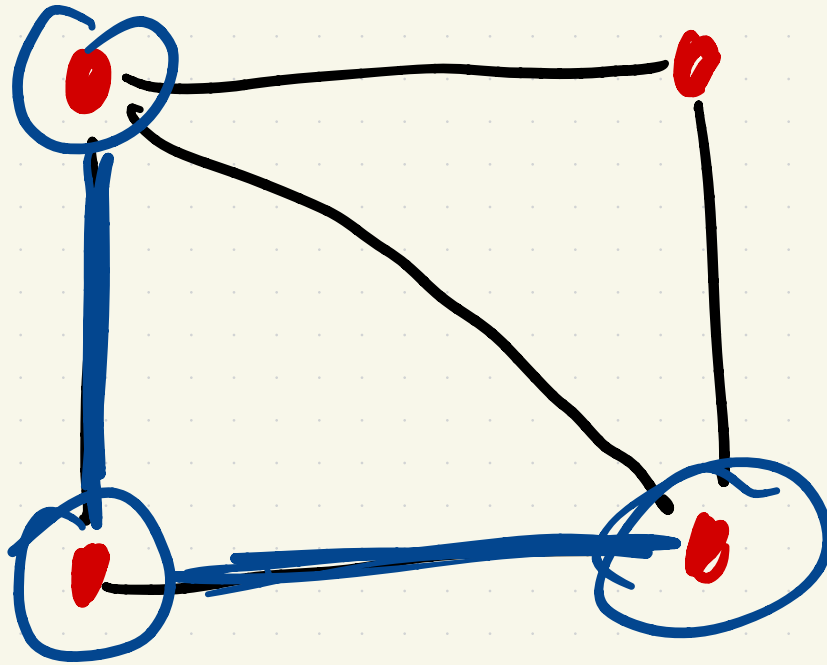
Until there are $|V|-1$ edges:

- Add the shortest edge that has one node inside the current MST.

Graph cut interpretation (why it works):

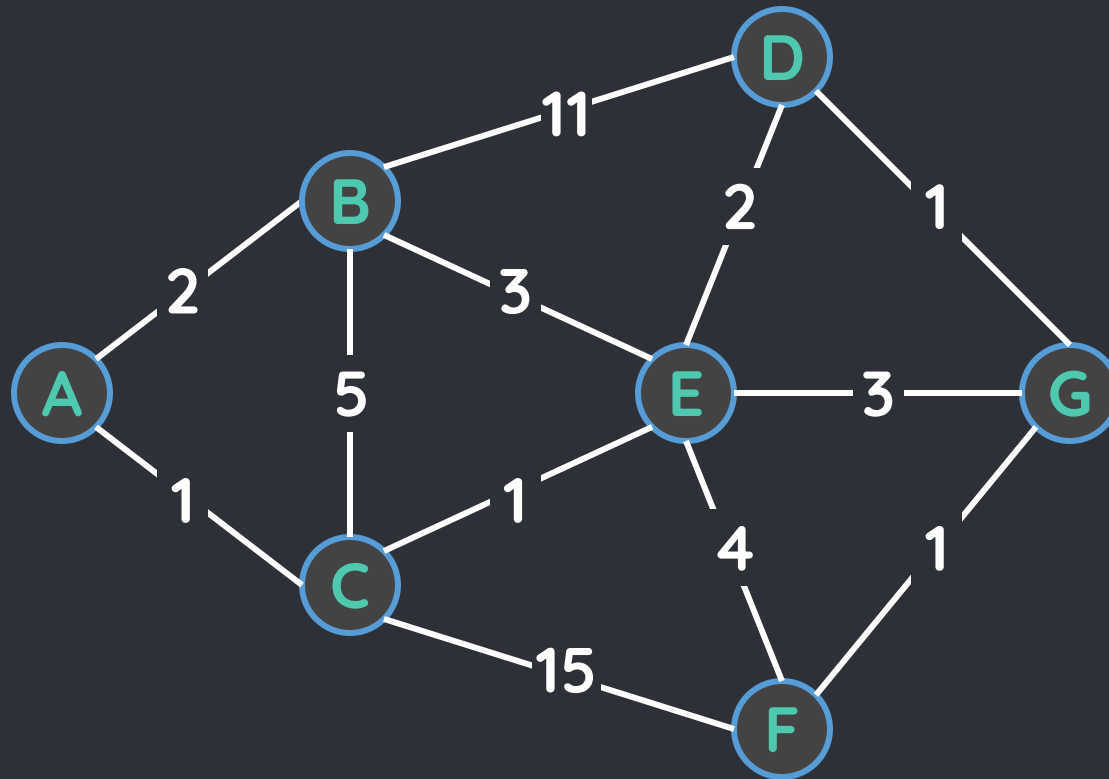
- For each iteration, we set the graph cut to be the nodes in the current MST, and its complement.
- The shortest edge selected is the minimum weight crossing edge.

start

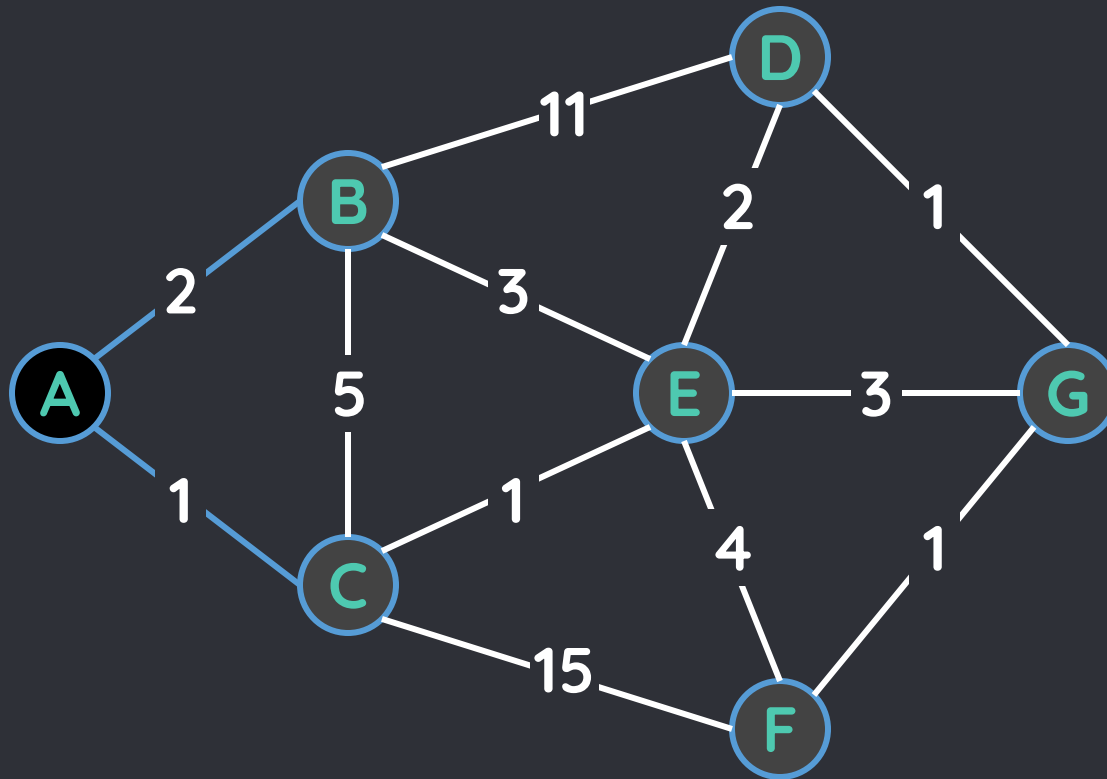


Circle blue - part of MST

- Prim's Algorithm - Concept

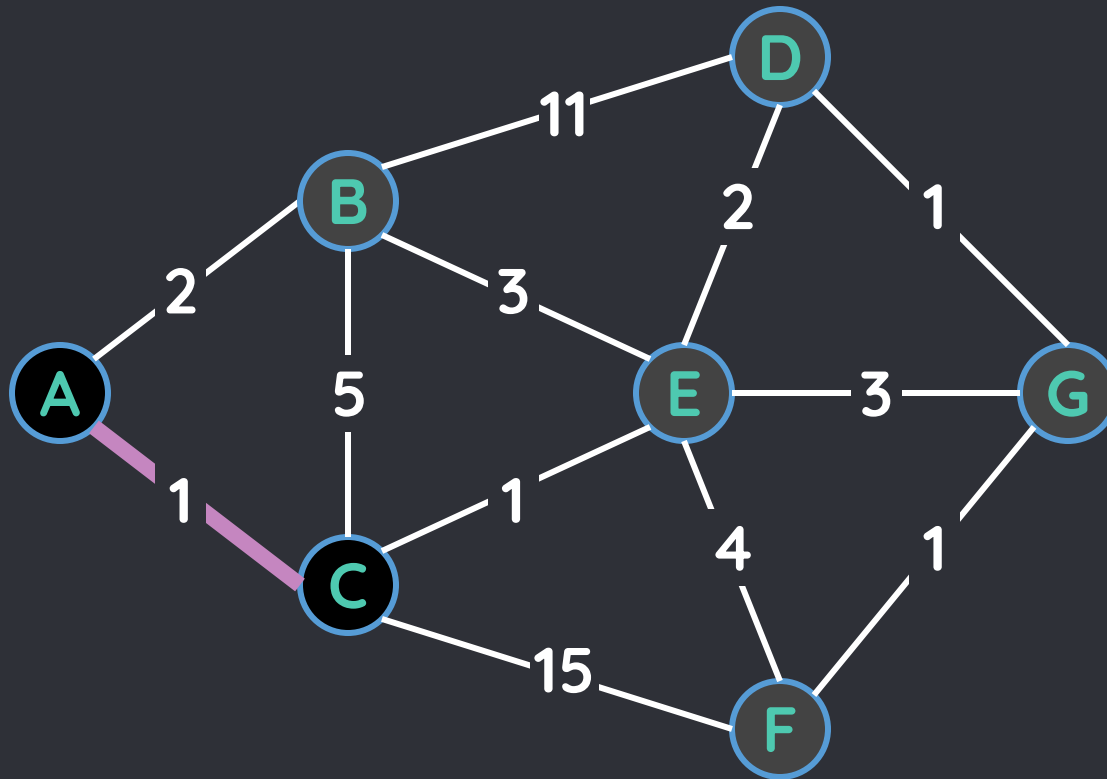


- Prim's Algorithm - Concept

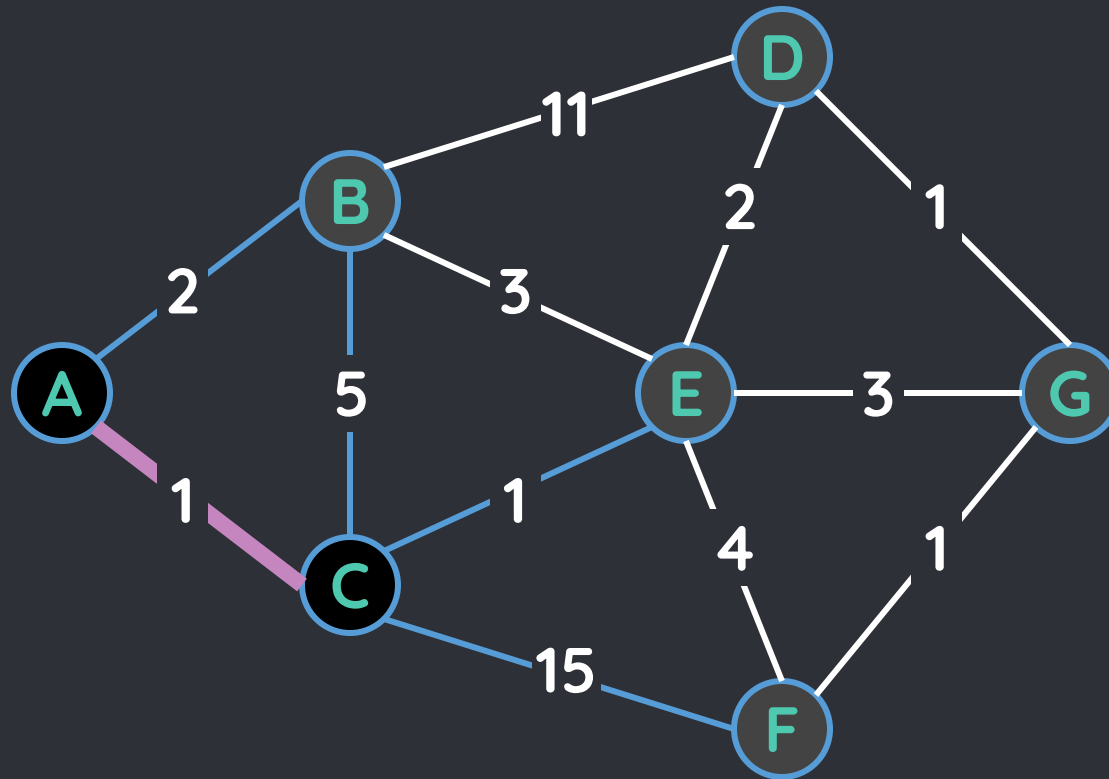


Start from any
arbitrary node, e.g. A

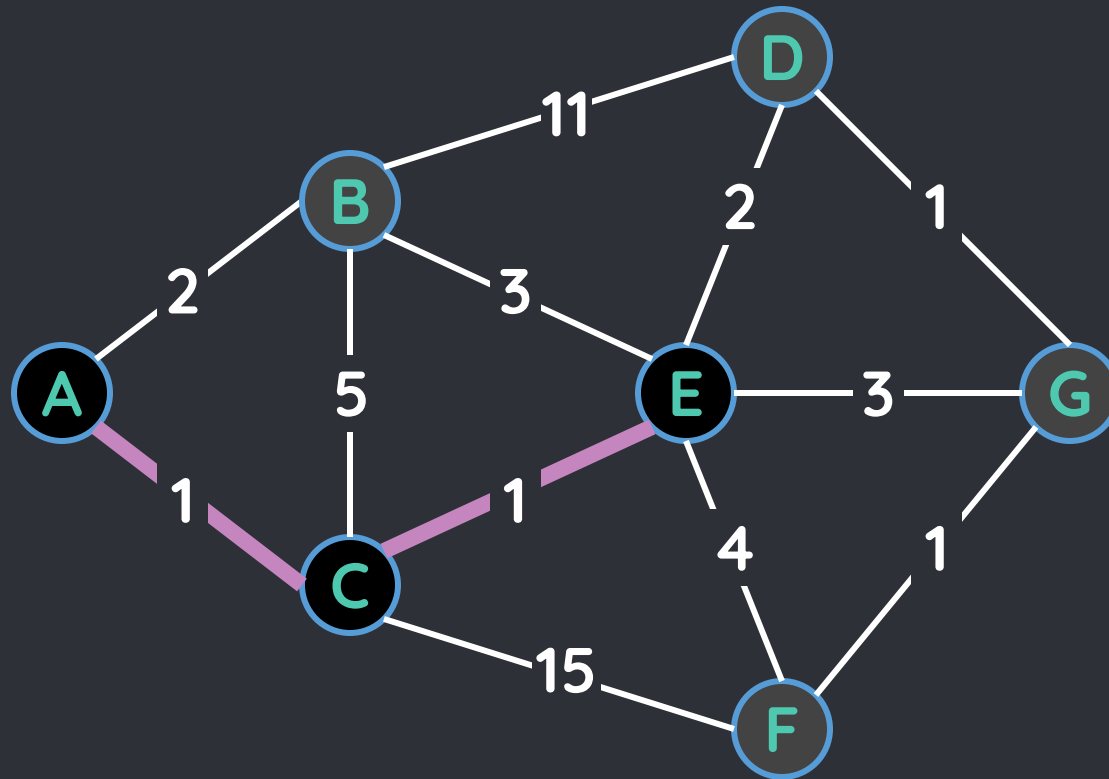
- Prim's Algorithm - Concept



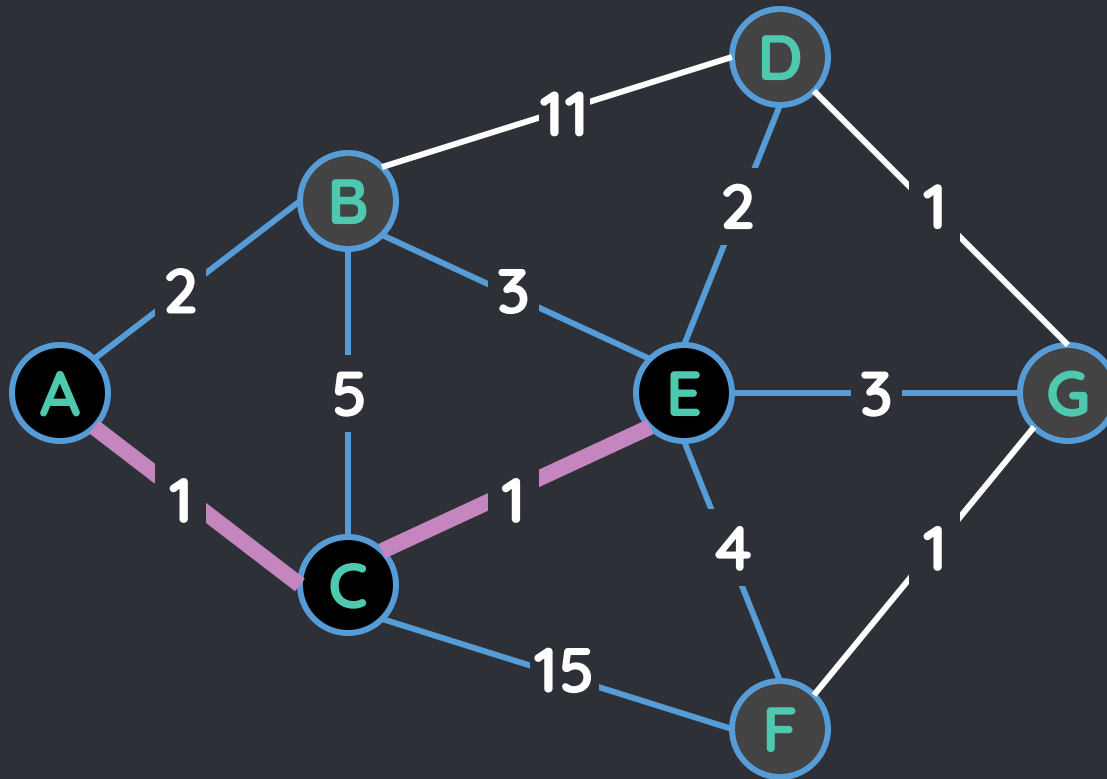
- Prim's Algorithm - Concept



- Prim's Algorithm - Concept

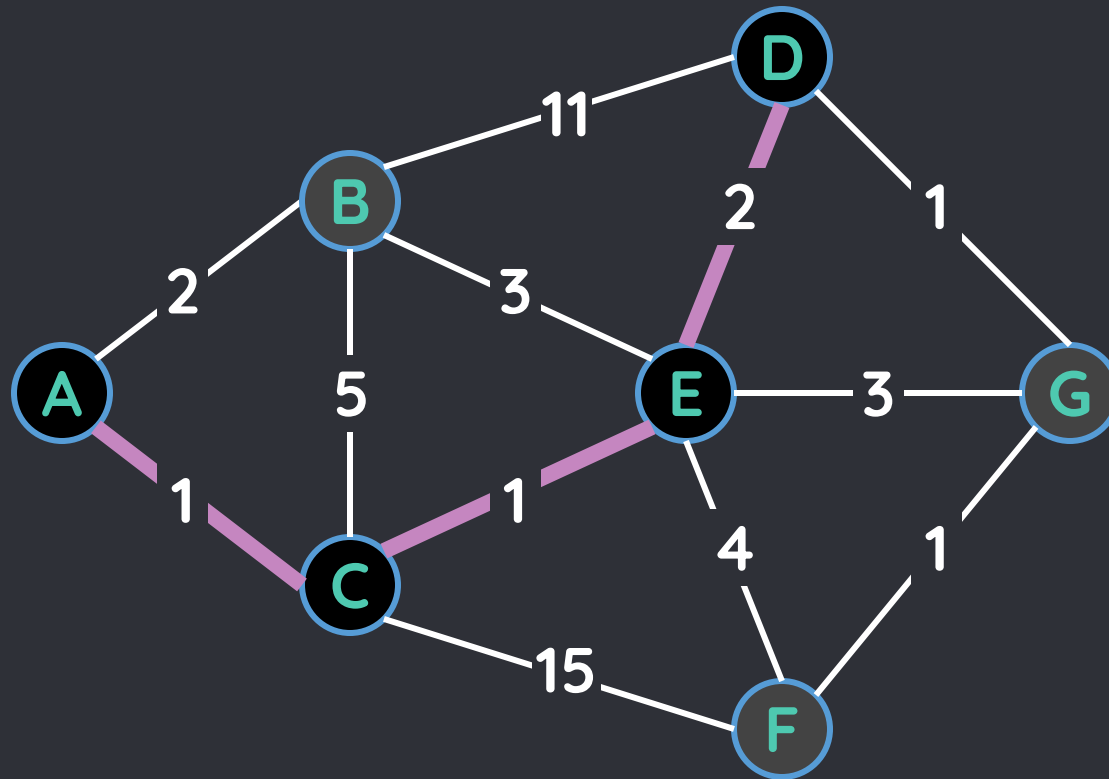


- Prim's Algorithm - Concept

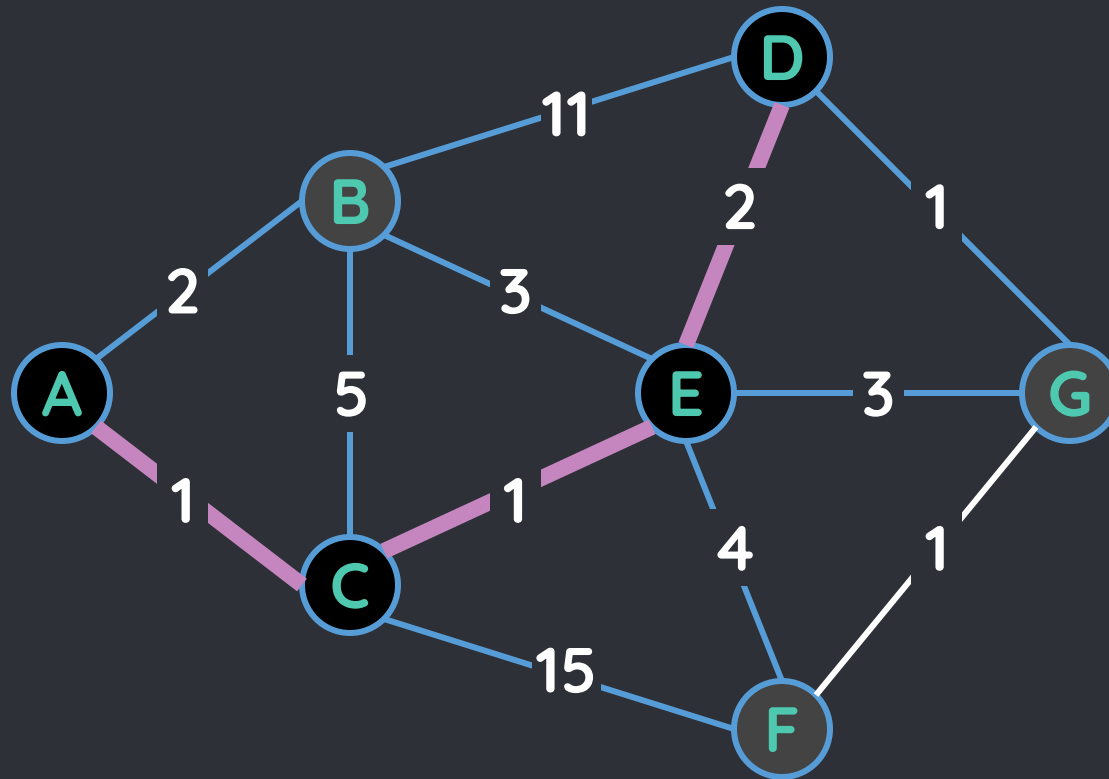


When there are multiple minimum crossing edges, chose any.

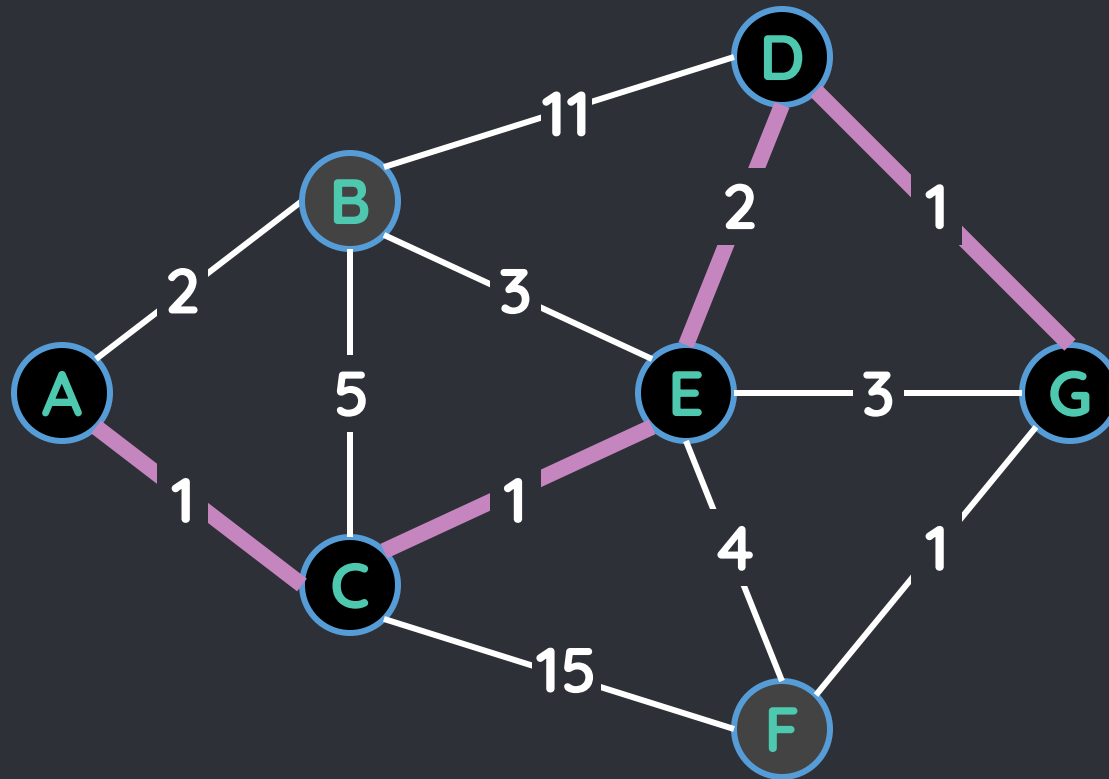
- Prim's Algorithm - Concept



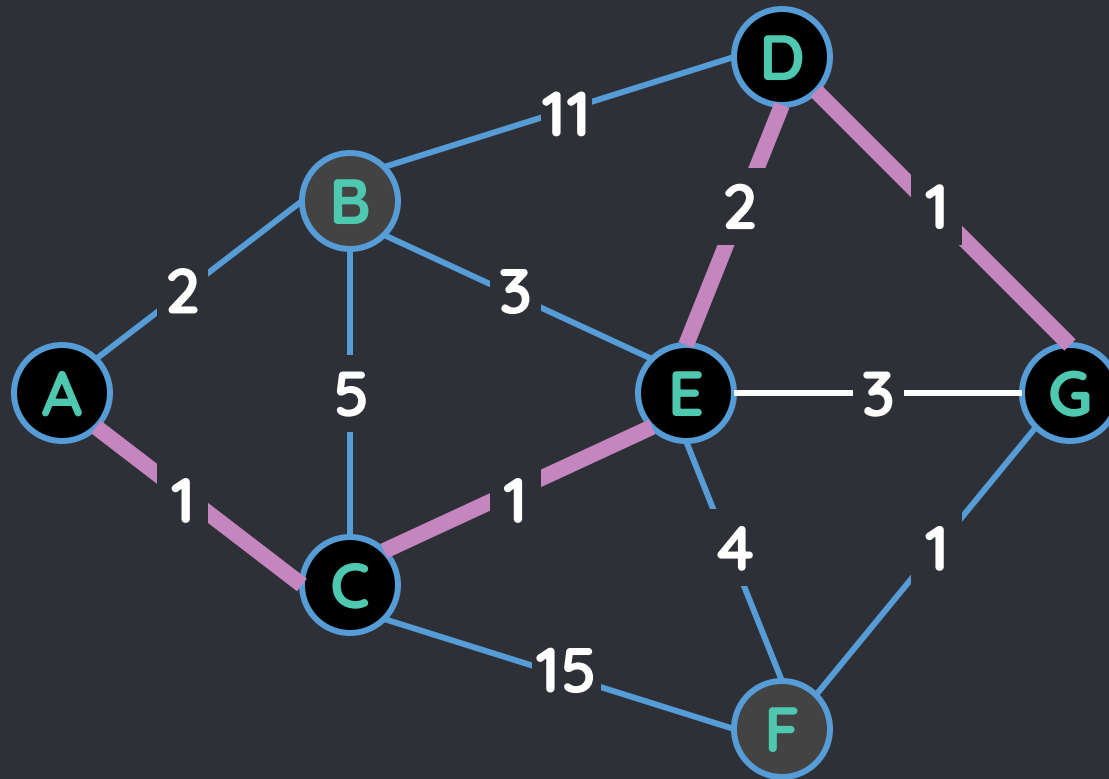
- Prim's Algorithm - Concept



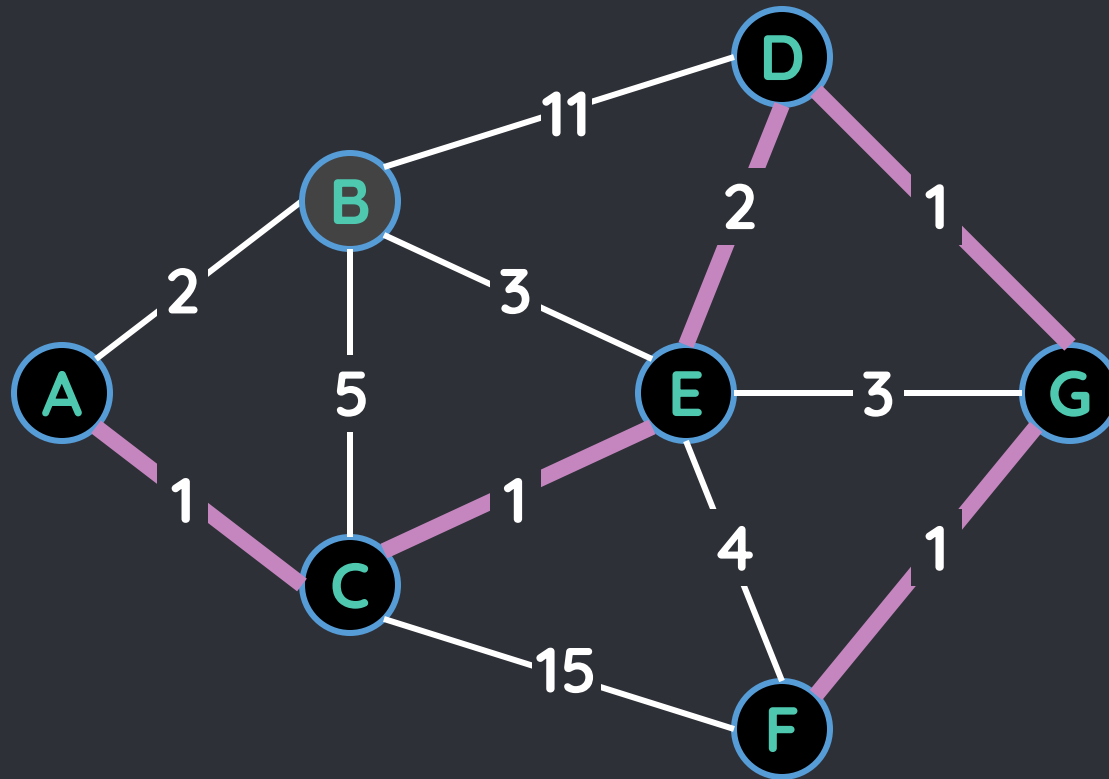
- Prim's Algorithm - Concept



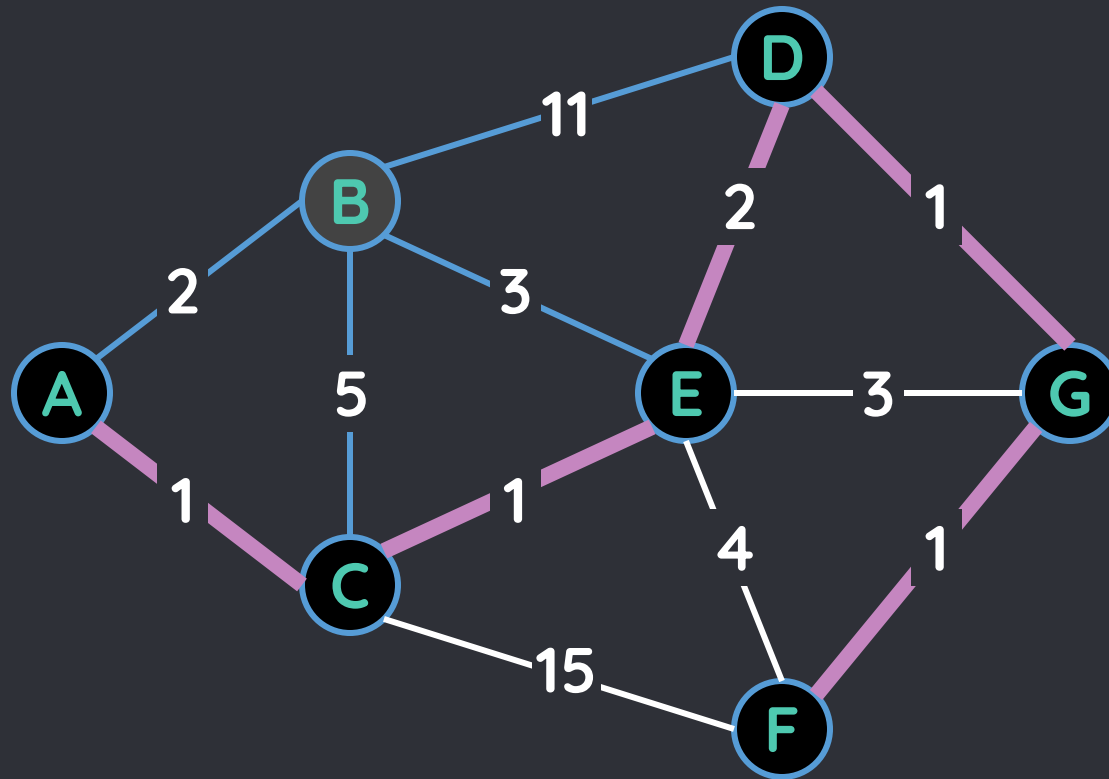
- Prim's Algorithm - Concept



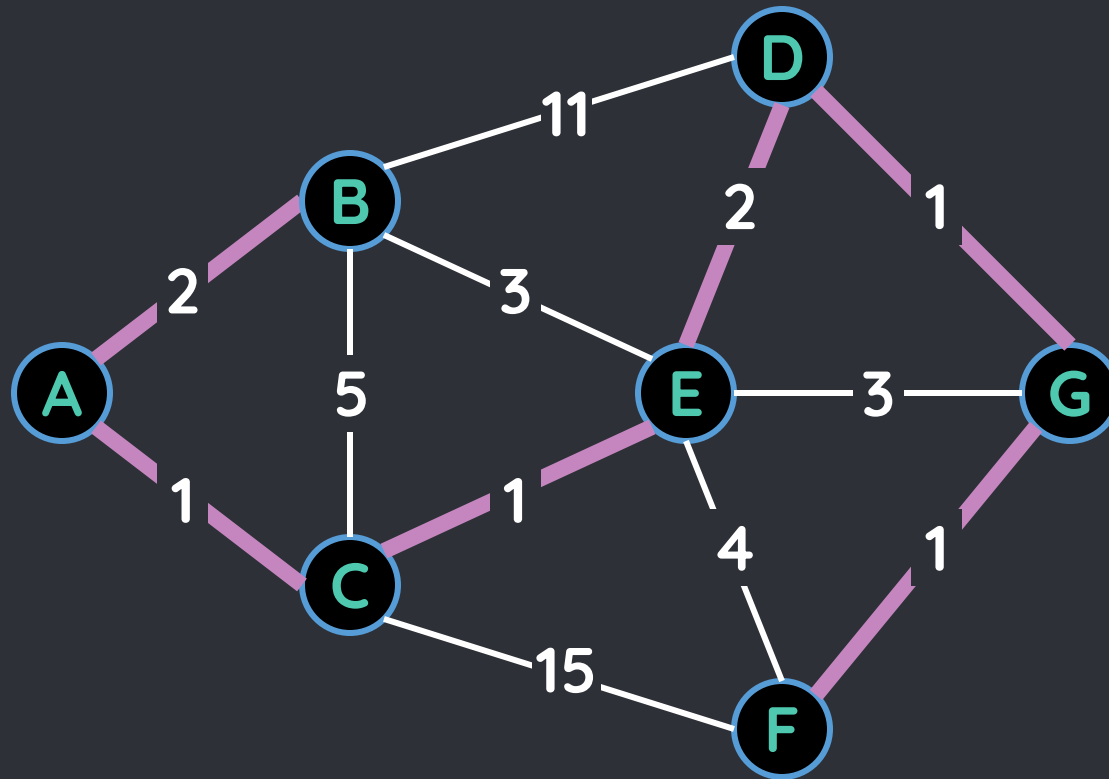
- Prim's Algorithm - Concept



- Prim's Algorithm - Concept



- Prim's Algorithm - Concept



• Prim's Algorithm - Optimized

What is the problem with the Prim's algorithm concept?

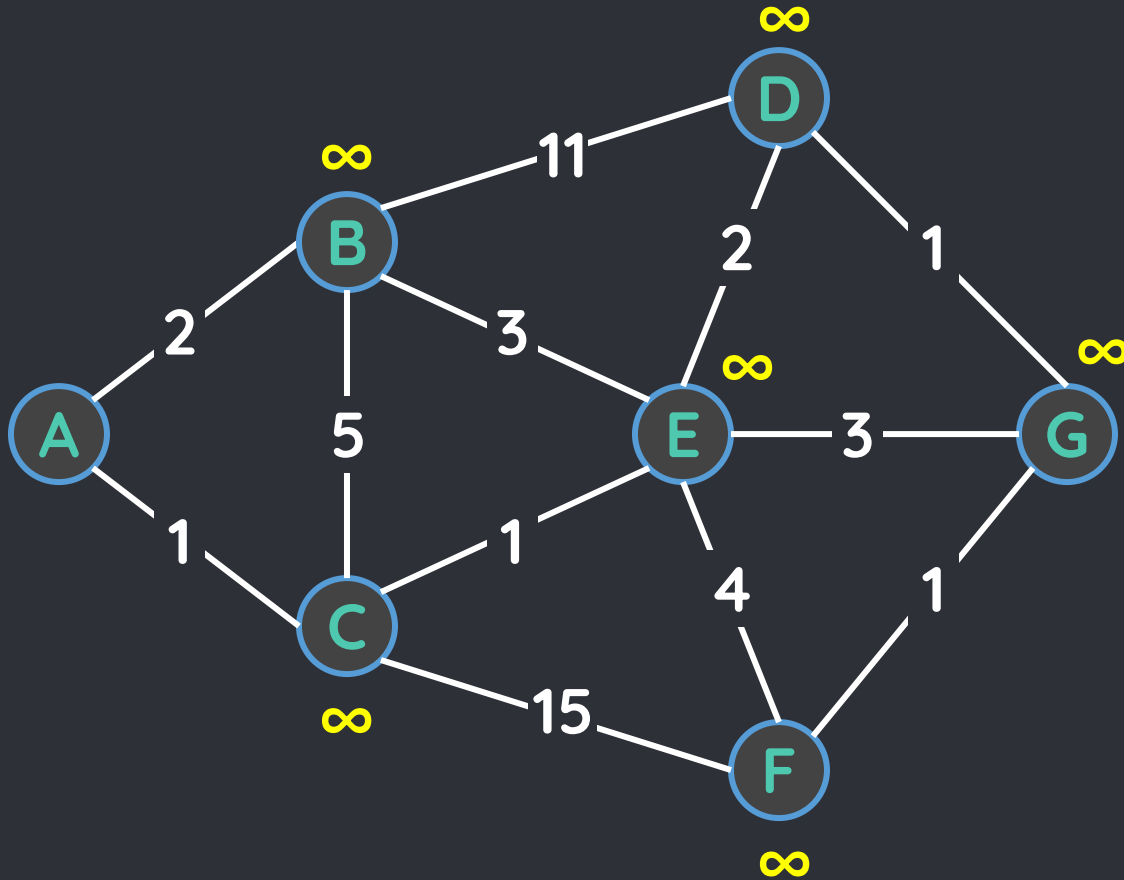
Problem:

- For every iteration:
 - need to find crossing edges
 - need to determine minimum edge

Solution:

- Store the distance of each node from the current MST (crossing edge length) in a priority queue.

- Prim's Algorithm

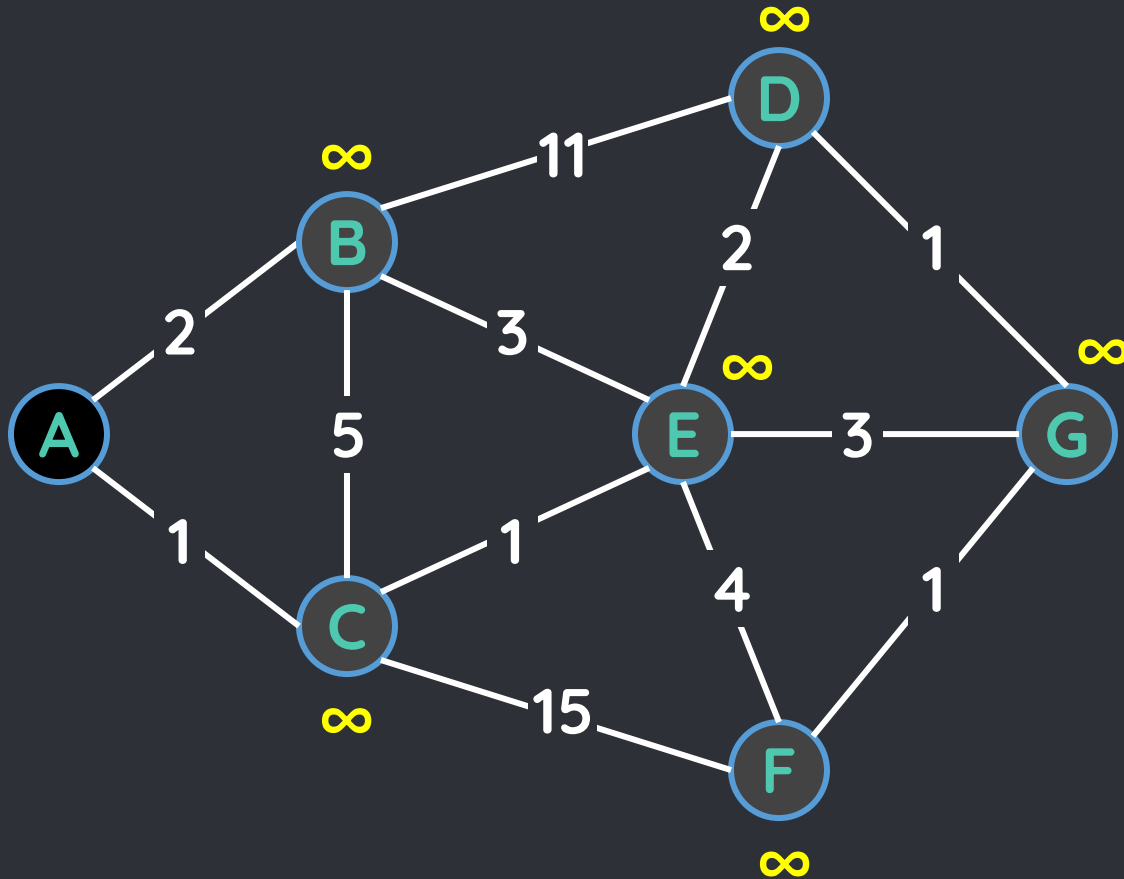


v	$d_{MST}[v]$	$edg[v]$
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

PQ: []

• Prim's Algorithm

Select an arbitrary starting node and include it in the MST

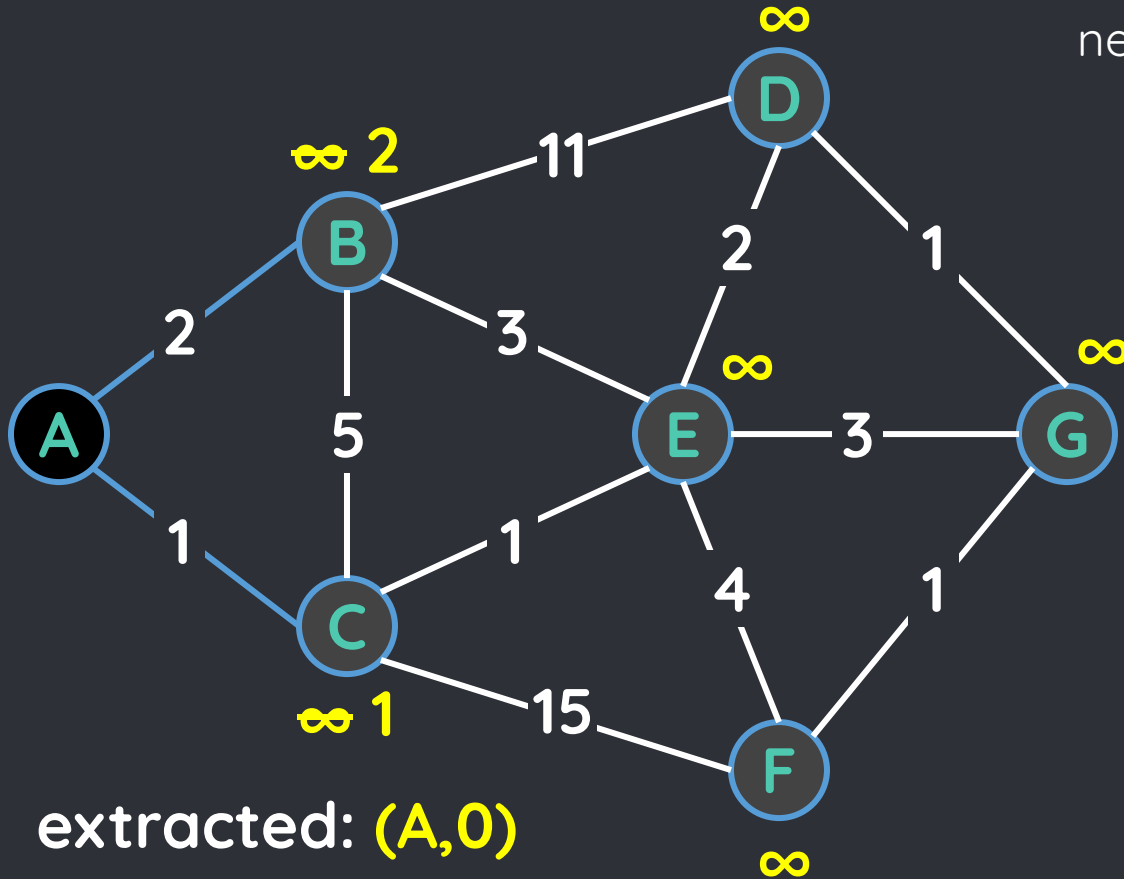


v	$d_{MST}[v]$	$edg[v]$
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

PQ: [(A,0)]

• Prim's Algorithm

Pop the front of the PQ
Compute the distance of the neighbors from popped node



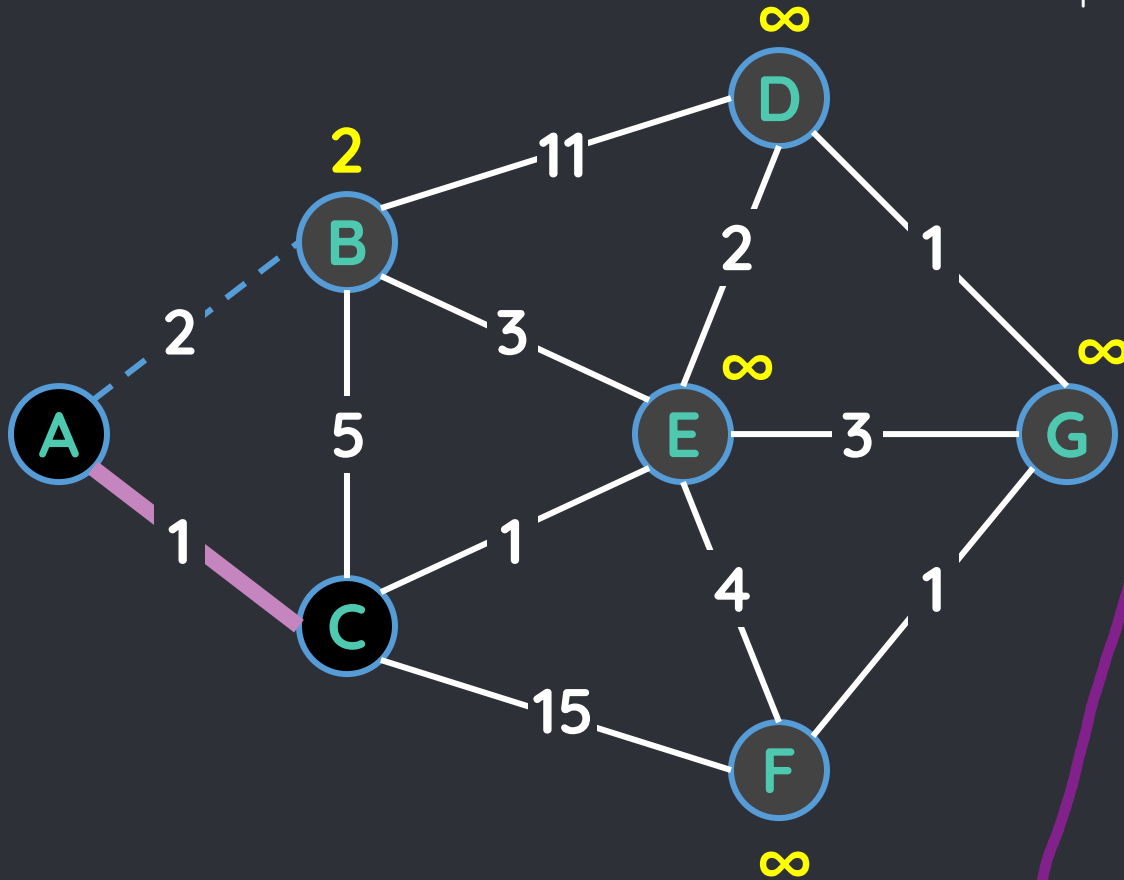
extracted: (A,0)

PQ: [(C,1), (B,2)]

v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	∞ 2	A
C	∞ 1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-

• Prim's Algorithm

Add vertex with minimum priority to the MST along with the corresponding edge



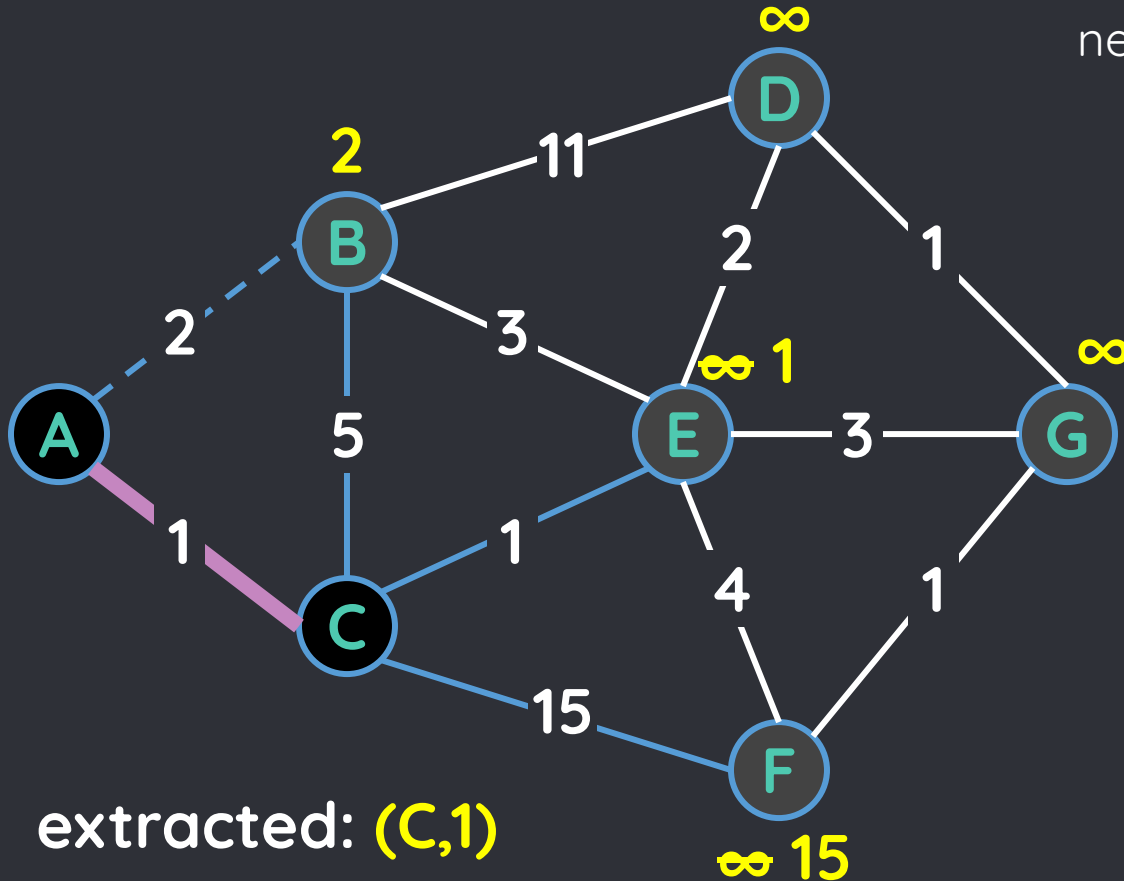
PQ: [(C,1), (B,2)]

v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	2	A
C	0	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-

changed to 0 since C is now part of the MST

• Prim's Algorithm

Pop the front of the PQ
Compute the distance of the neighbors from popped node



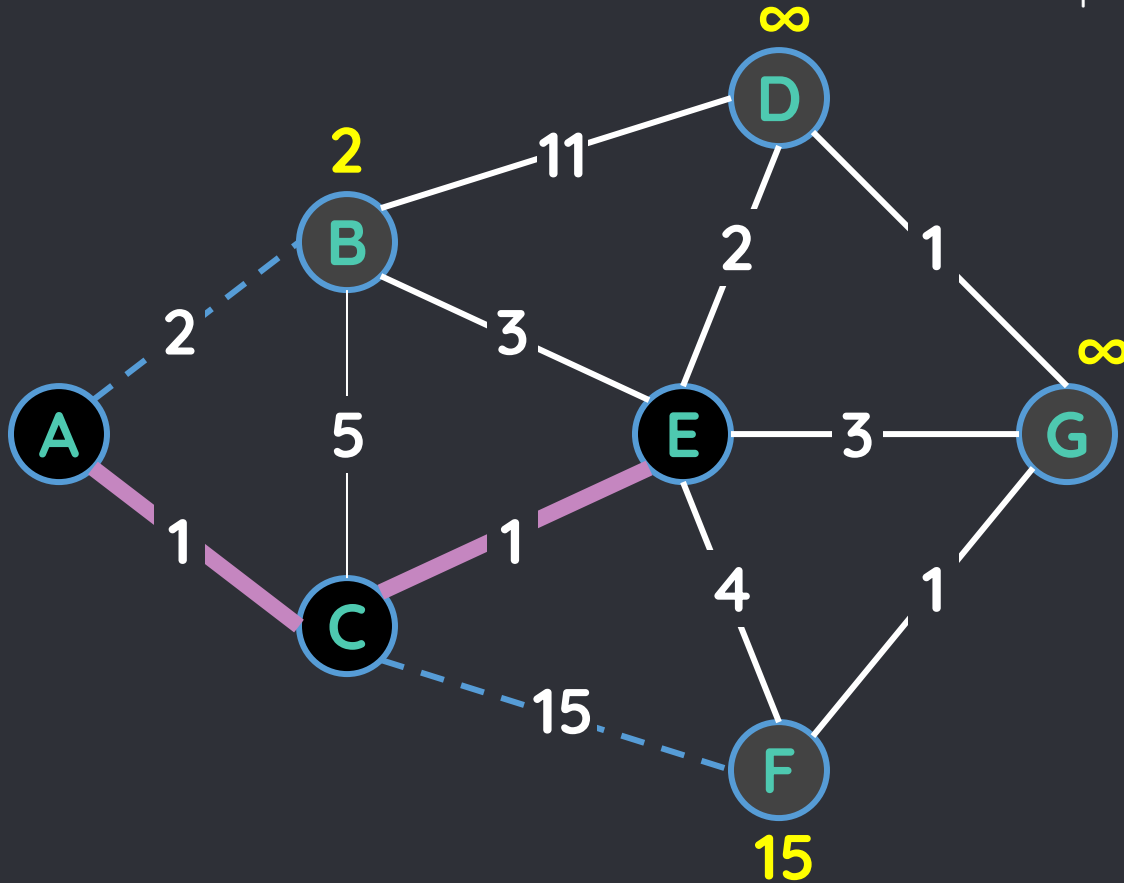
extracted: (C,1)

PQ: [(E,1), (B,2), (F,15)]

v	$d_{MST}[v]$	edg[v]
A	0	-
B	2	A
C	0	A
D	∞	-
E	∞ 1	C
F	∞ 15	C
G	∞	-

• Prim's Algorithm

Add vertex with minimum priority to the MST along with the corresponding edge

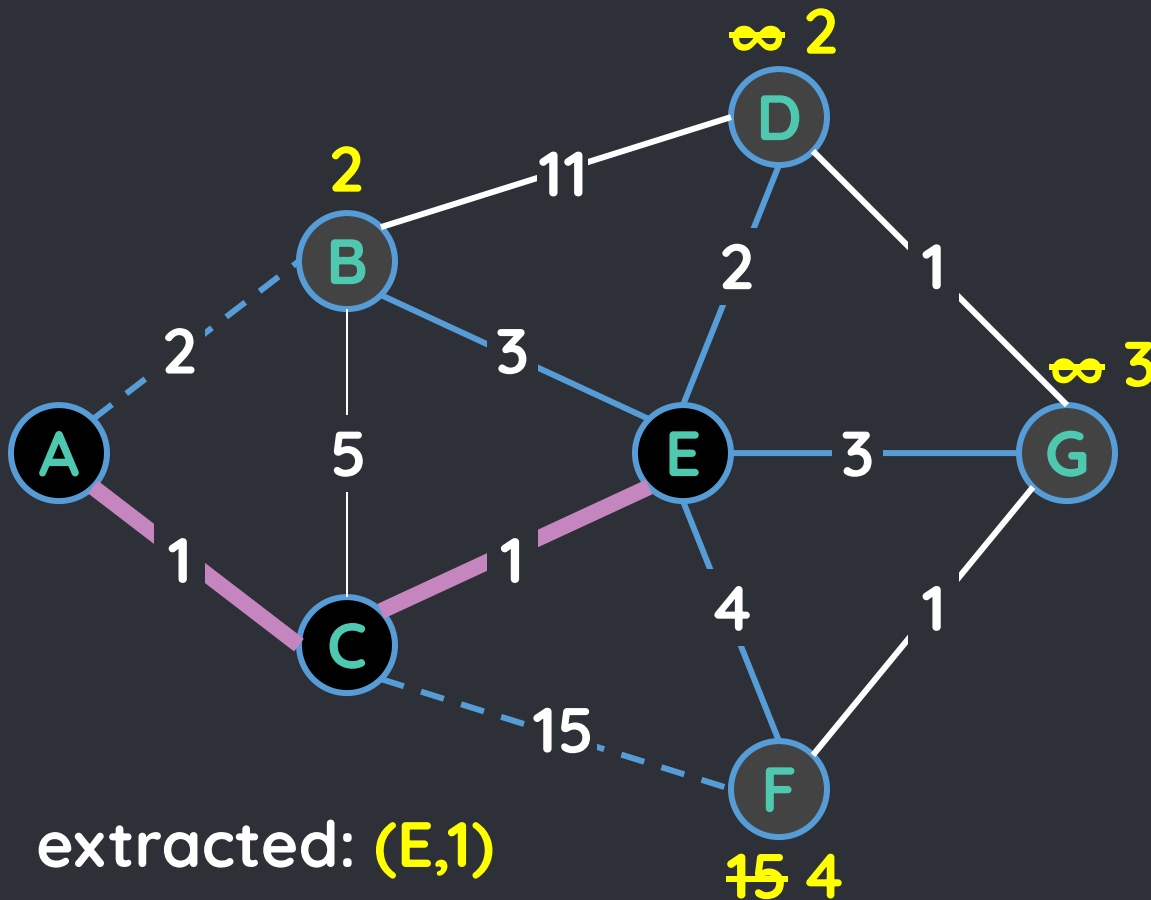


v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	2	A
C	0	A
D	∞	-
E	0	C
F	15	C
G	∞	-

PQ: [(E,1), (B,2), (F,15)]

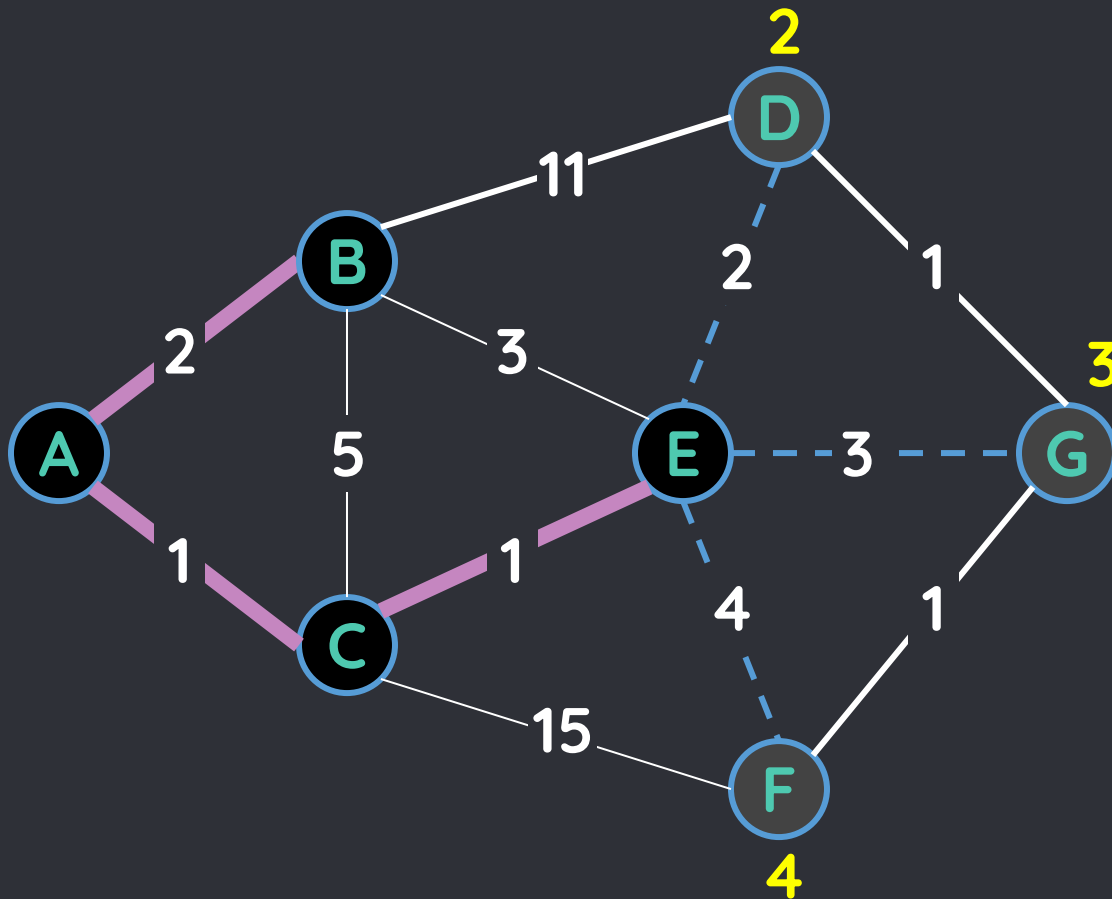
• Prim's Algorithm

... repeat until $|V| - 1$ edge or all vertices are in the MST



v	$d_{MST}[v]$	$edg[v]$
A	0	-
B	2	A
C	0	A
D	∞ 2	E
E	0	C
F	15 4	$\in E$
G	∞ 3	E

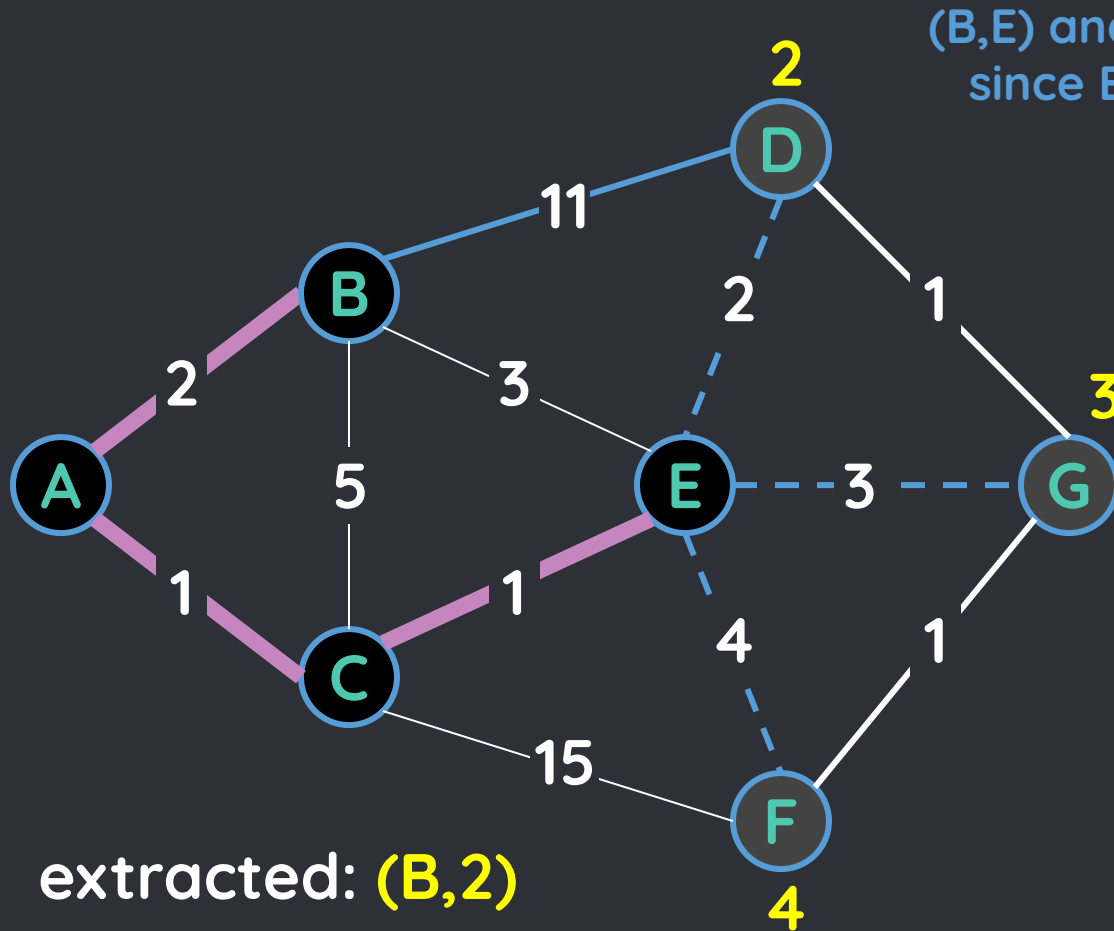
- Prim's Algorithm



v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	0	A
C	0	A
D	2	E
E	0	C
F	4	E
G	3	E

PQ: [(B,2), (D,2), (G,3), (F,4)]

• Prim's Algorithm

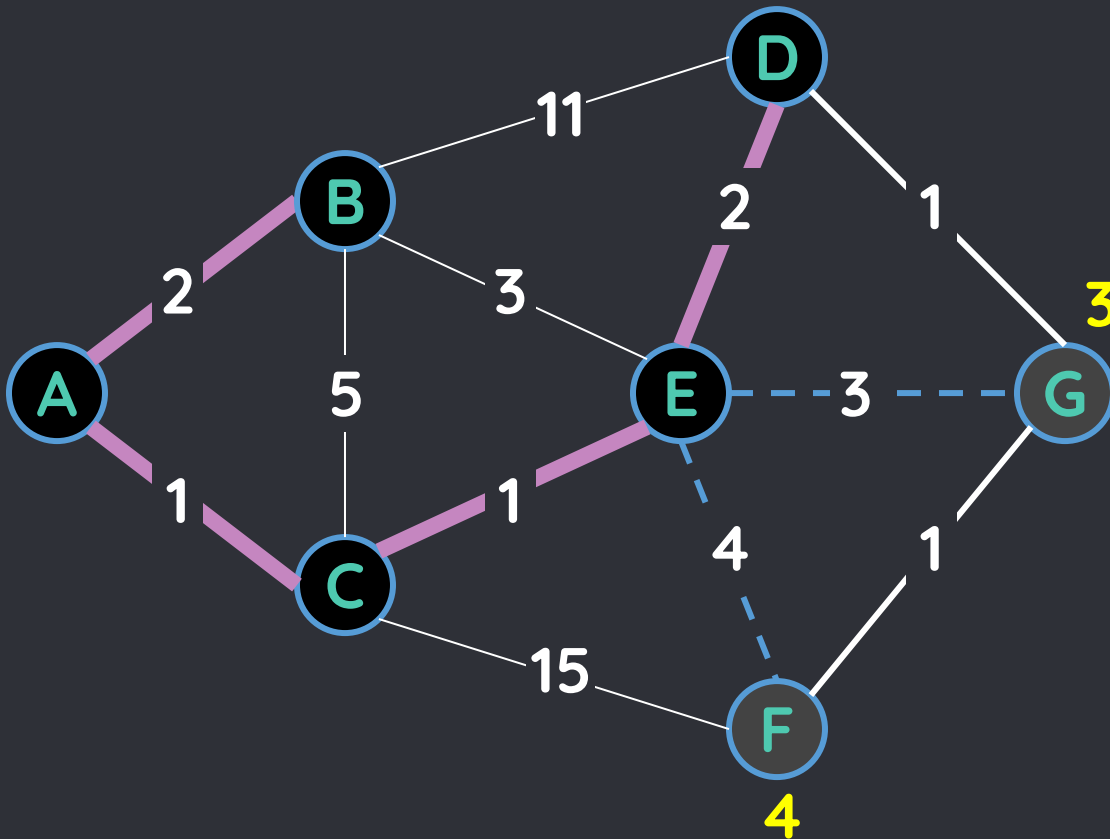


v	$d_{MST}[v]$	$pre[v]$
A	0	-
B	0	A
C	0	A
D	2	E
E	0	C
F	4	E
G	3	E

extracted: (B,2)

PQ: [(D,2), (G,3), (F,4)]

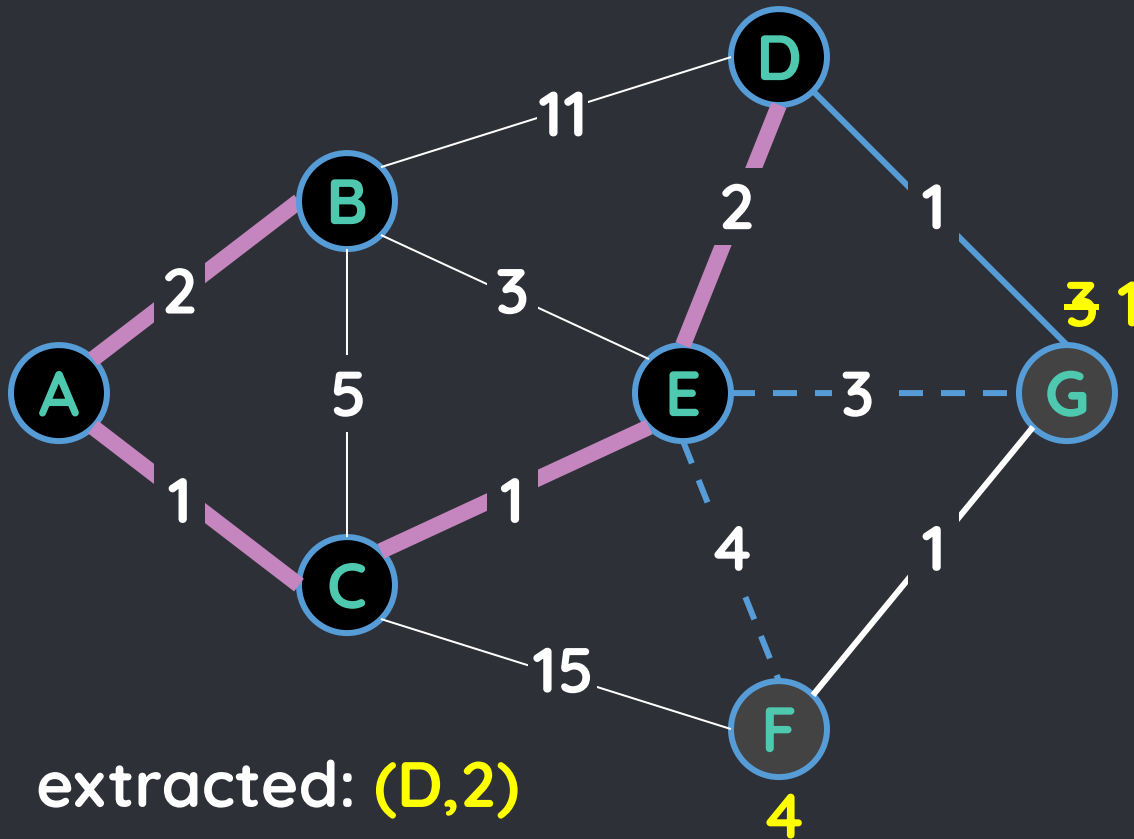
- Prim's Algorithm



v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	0	A
C	0	A
D	0	E
E	0	C
F	4	E
G	3	E

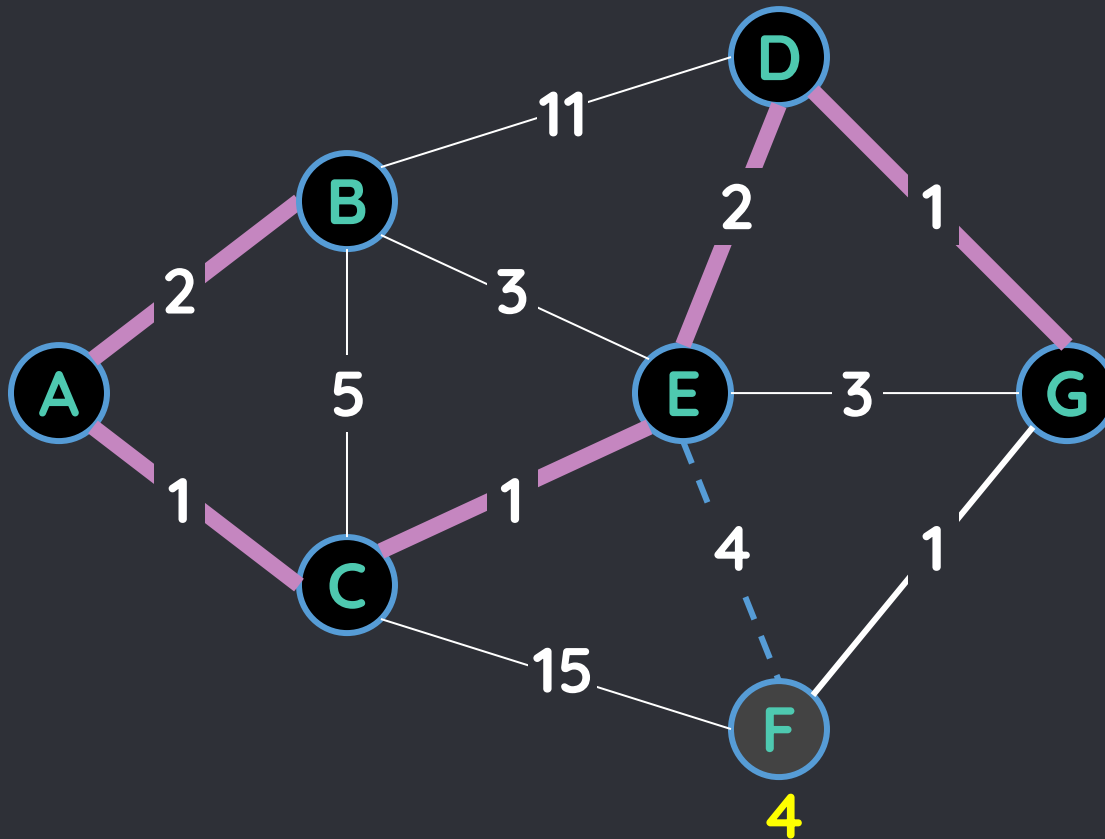
PQ: [(D,2), (G,3), (F,4)]

- Prim's Algorithm



v	$d_{MST}[v]$	edg[v]
A	0	-
B	0	A
C	0	A
D	0	E
E	0	C
F	4	E
G	3 1	E D

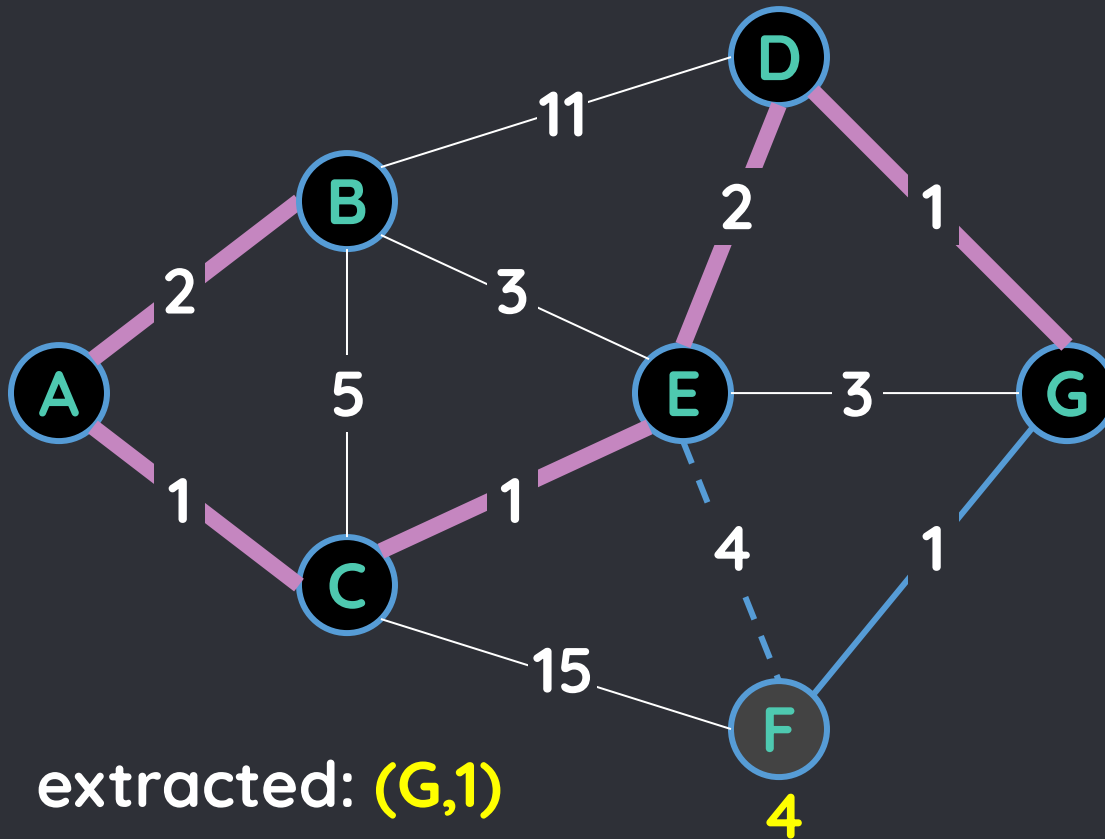
- Prim's Algorithm



v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	0	A
C	0	A
D	0	E
E	0	C
F	4	E
G	0	D

PQ: [(G,1), (F,4)]

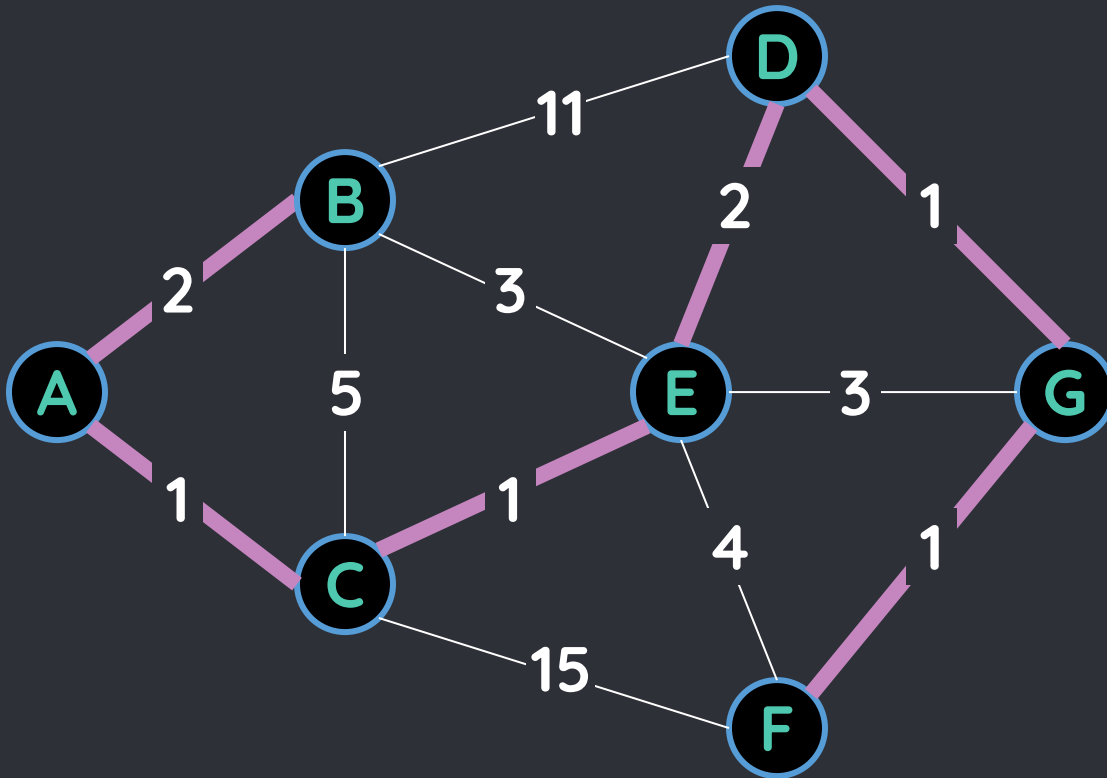
- Prim's Algorithm



v	$d_{MST}[v]$	$edge[v]$
A	0	-
B	0	A
C	0	A
D	0	E
E	0	C
F	4 1	E G
G	0	D

- Prim's Algorithm

Construct MST



v	$d_{MST}[v]$	$edg[v]$
A	0	-
B	0	A
C	0	A
D	0	E
E	0	C
F	0	G
G	0	D

PQ: [(F,1)]

all nodes are in MST,
 $|V|-1$ edges $\rightarrow$ done

• Prim's Algorithm Complexity

Space complexity:

- $O(|V|)$ for distance and edge arrays

Time complexity:

- $O((|V|+|E|) \log |V|)$ [optimized]

↳ iterate over all vertices + edges
↳ vertices and edges are only checked
once.

↳ Worst case is we always update
the priority queue $O(\log |V|)$

• Review of Last Lecture

○ Minimum Spanning Tree Problem:

- Find a spanning tree of a graph **G** such that total weight of edges is minimal

Connected, acyclic graph that includes all vertices in **G**

Algorithm #1: Prim's Algorithm

- Based on the (graph) cut property
- Set A = MST and Set B = the rest of the graph

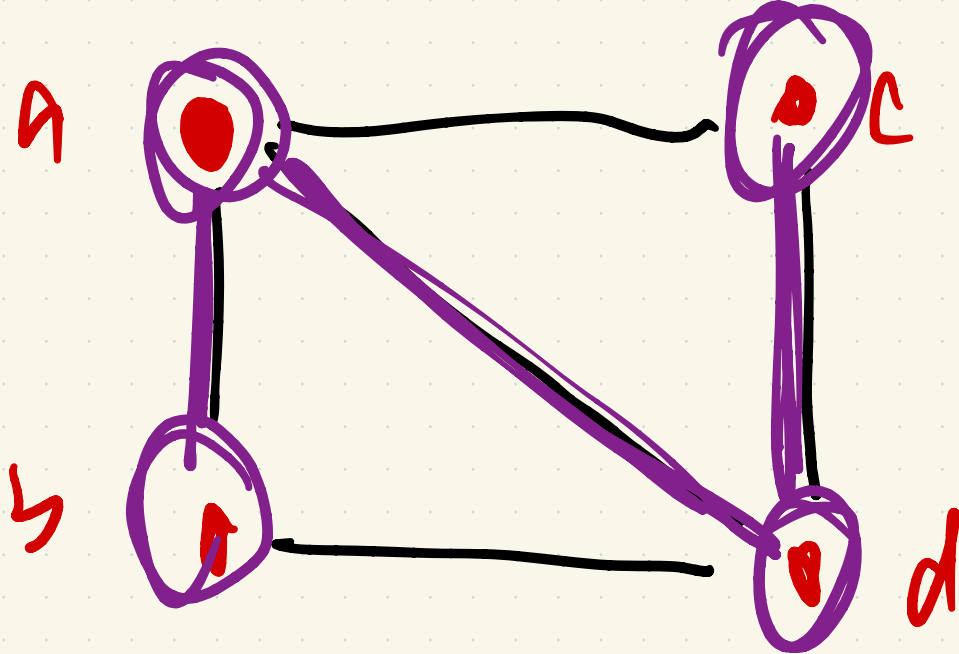
• Kruskal's Algorithm Concept

○ Until there are $|V|-1$ edges:

- Obtain a table of the sorted edges
- For each edge, add the edge with minimum weight to MST unless doing so creates a cycle

Graph cut interpretation (why it works):

- For each iteration, the graph cut contains all the vertices connected to one of the edge vertex, and its complement
- The next minimum edge is assured to be a minimum weight edge, **you just need to check if it is a crossing edge for it to be in the MST**



Does adding the edge create a cycle?

Edge	1	✓	$(a, d) = \text{findSet}(a), \text{findSet}(d)$
"	2	✓	
"	3	✗	
"	4	✓	
"	...	...	

● Kruskal's Algorithm Optimized

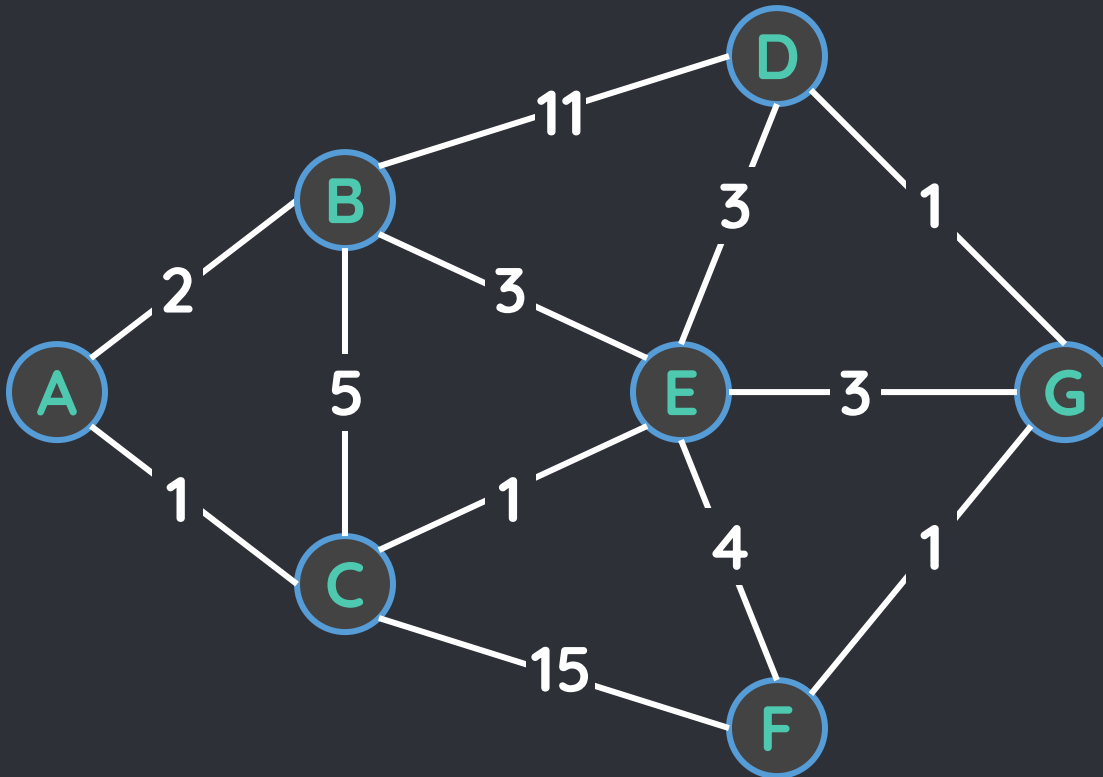
○ Store the vertices in a disjoint set (e.g. WQUPC)

Store the edges in a min PQ (e.g. min heap)

Iterate over all the edges in PQ until there are $|V|-1$ edges:

- If $\text{find}(u,v)$ is true (i.e. cycle detected):
 - Ignore edge
- Else:
 - Add edge and corresponding vertex to MST
 - $\text{union}(u,v)$

- Kruskal's Algorithm Optimized

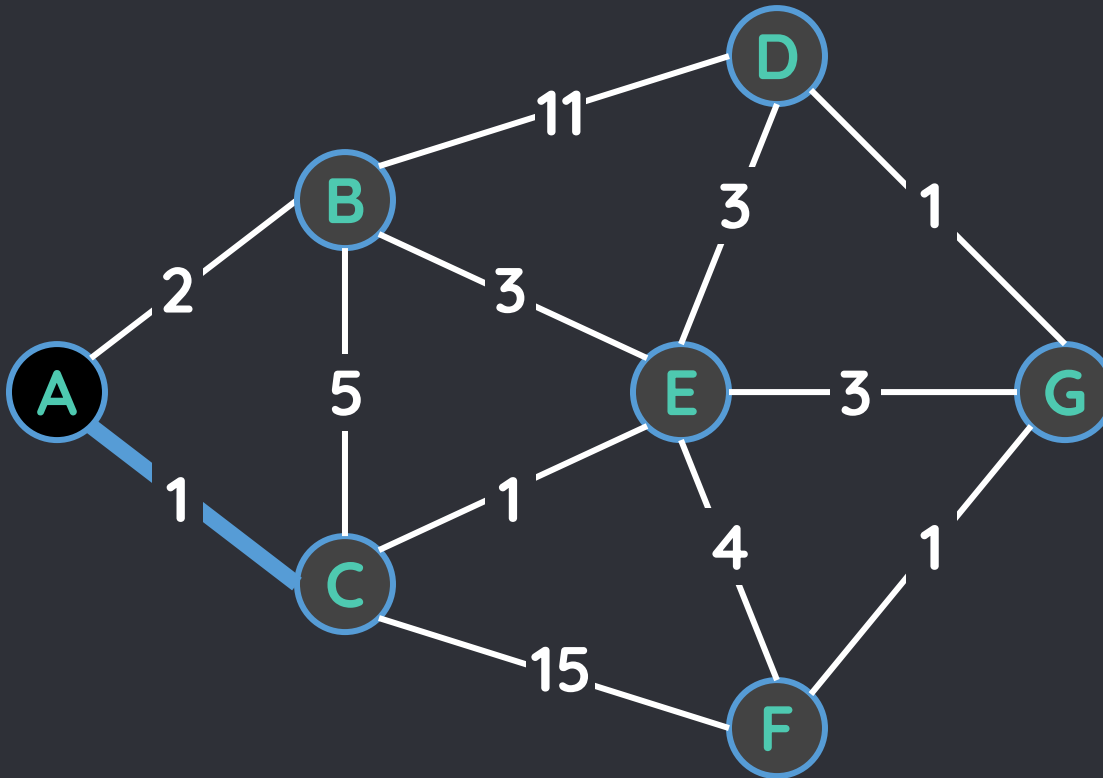


DS: [{A},{B},{C},{D},{E},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

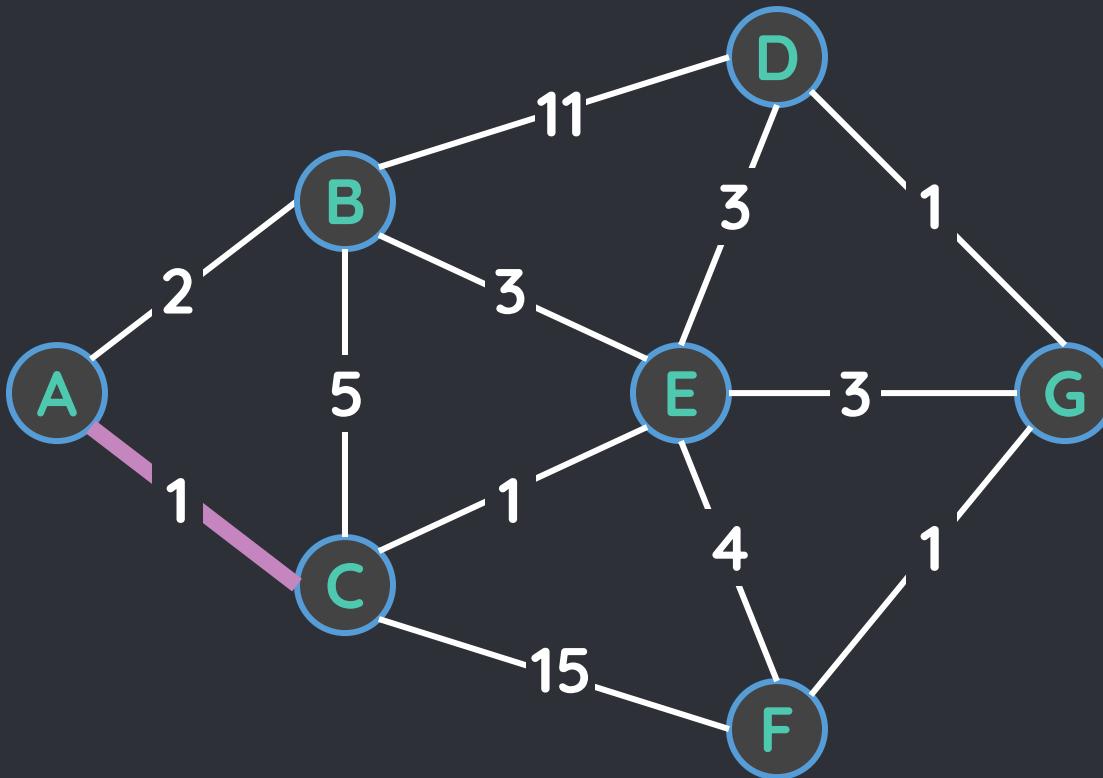
find(A,C) : False -> in MST



DS: [{A},{B},{C},{D},{E},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

- Kruskal's Algorithm Optimized

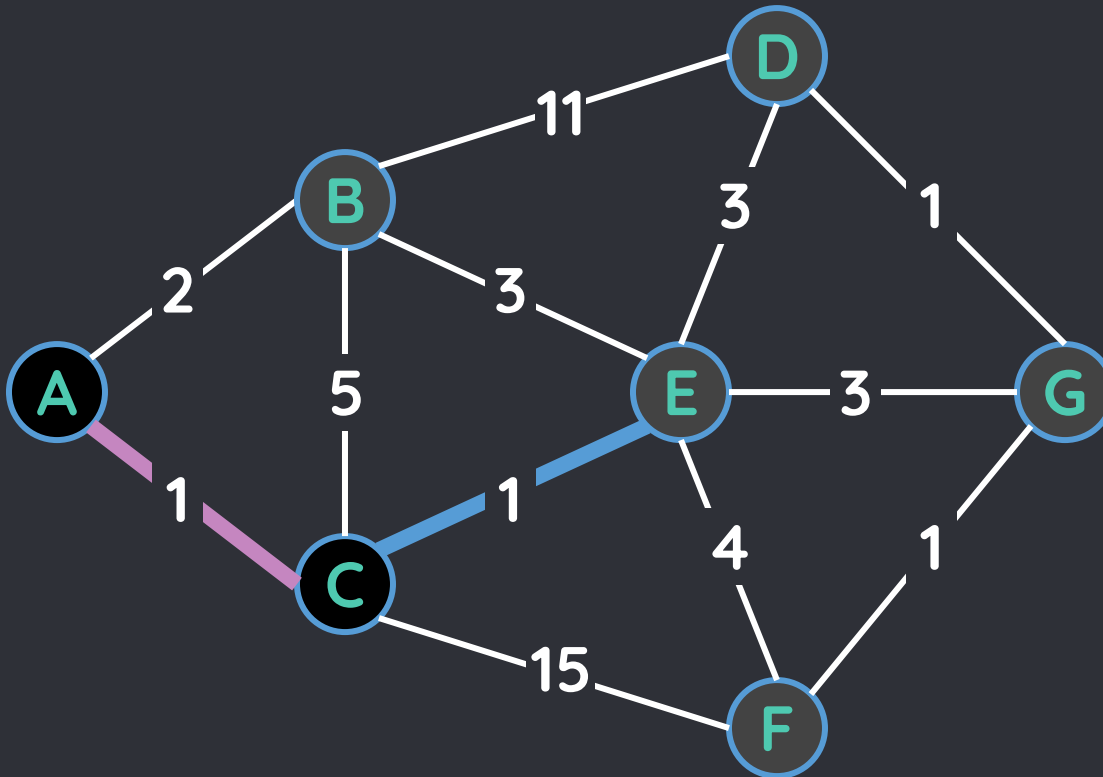


DS: [{A,C},{B},{D},{E},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

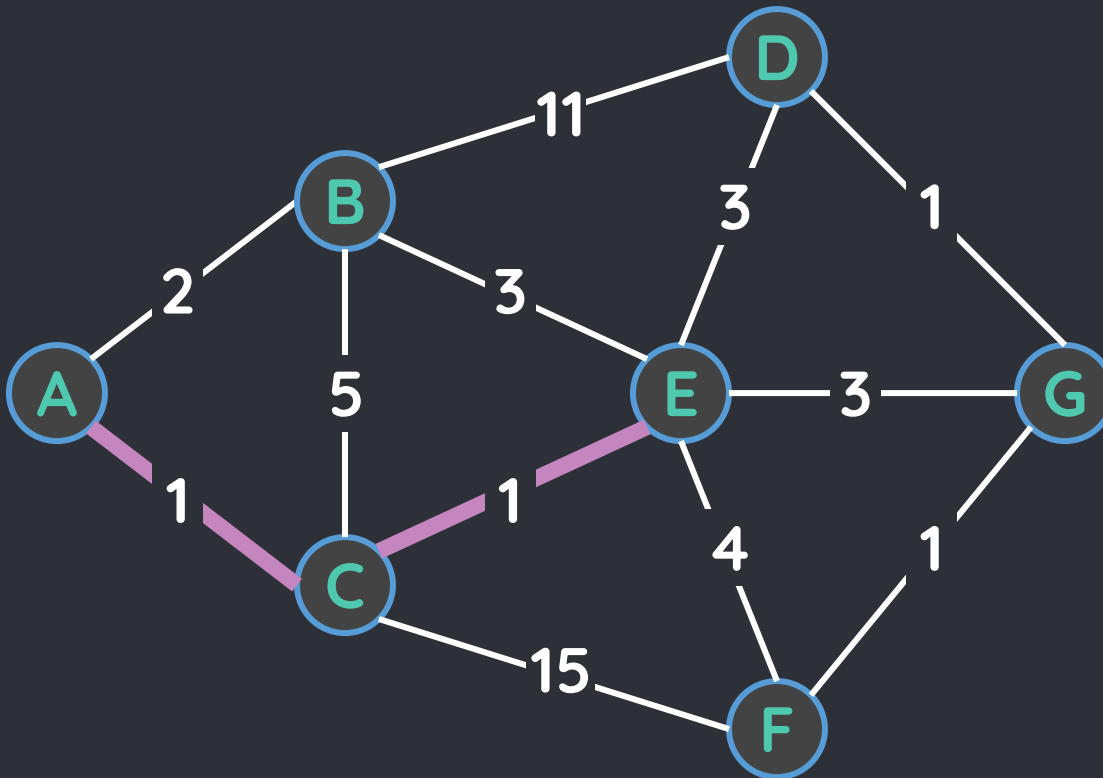
find(C,E) : False -> in MST



DS: [{A,C},{B},{D},{E},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

- Kruskal's Algorithm Optimized

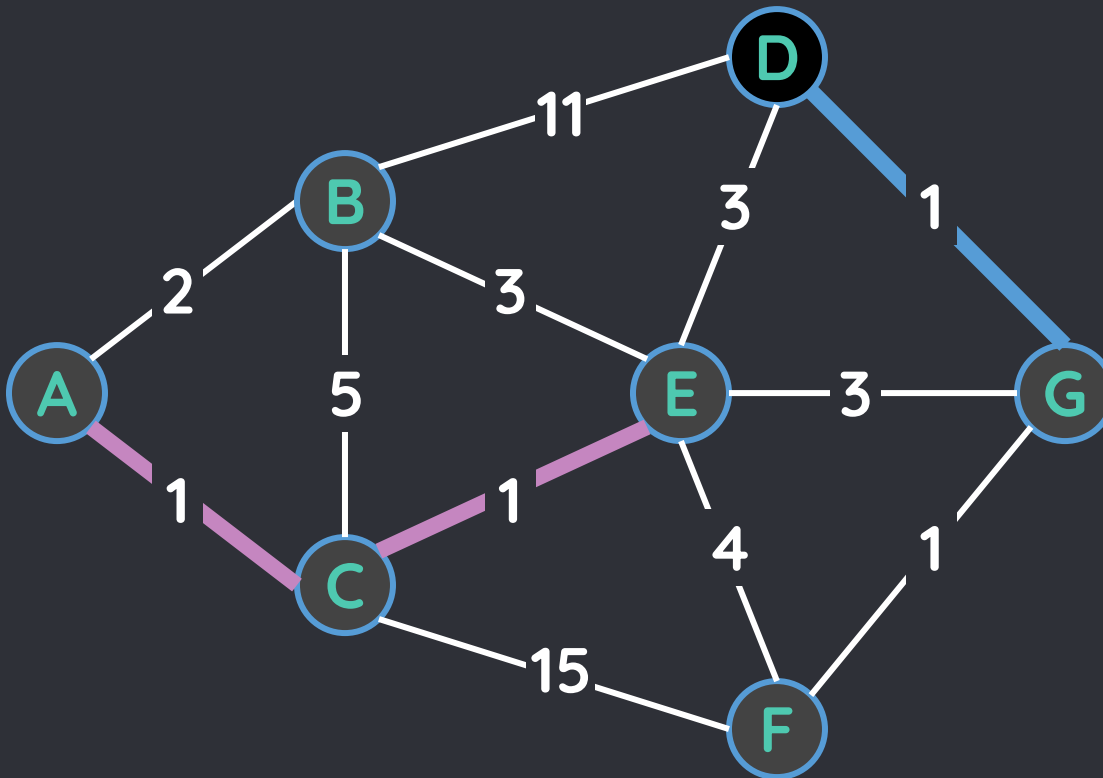


DS: [{A,C,E},{B},{D},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

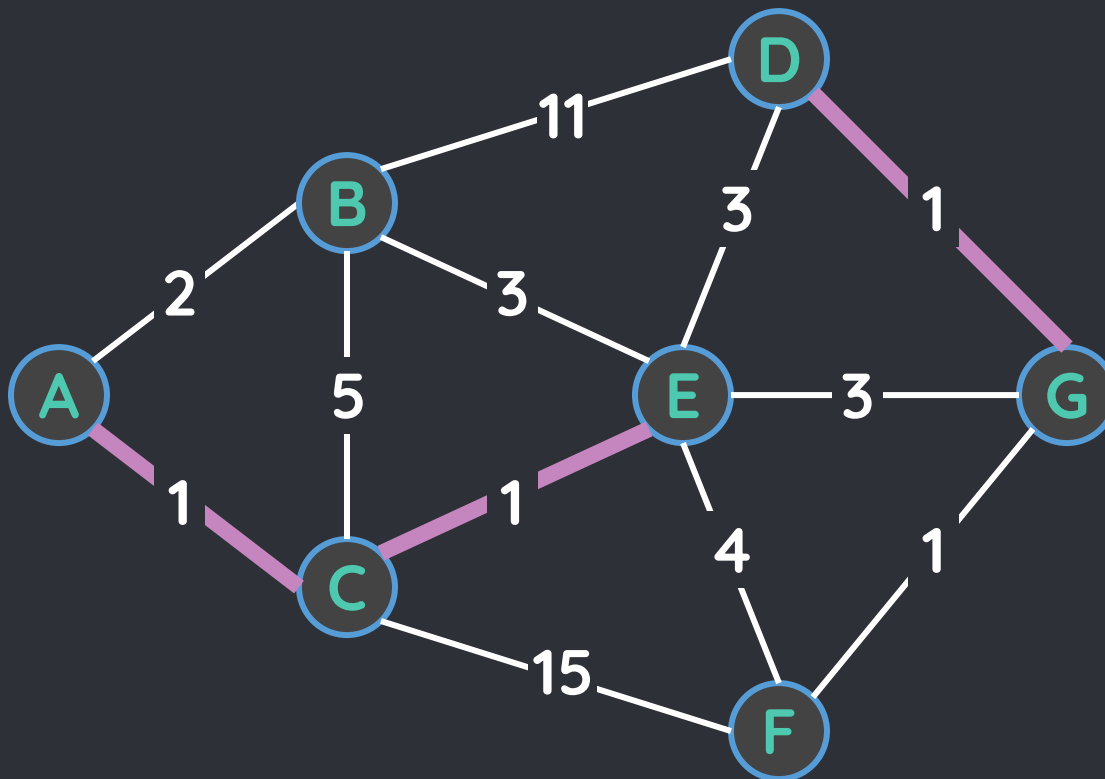
find(D,G) : False -> in MST



DS: [{A,C,E},{B},{D},{F},{G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

- Kruskal's Algorithm Optimized

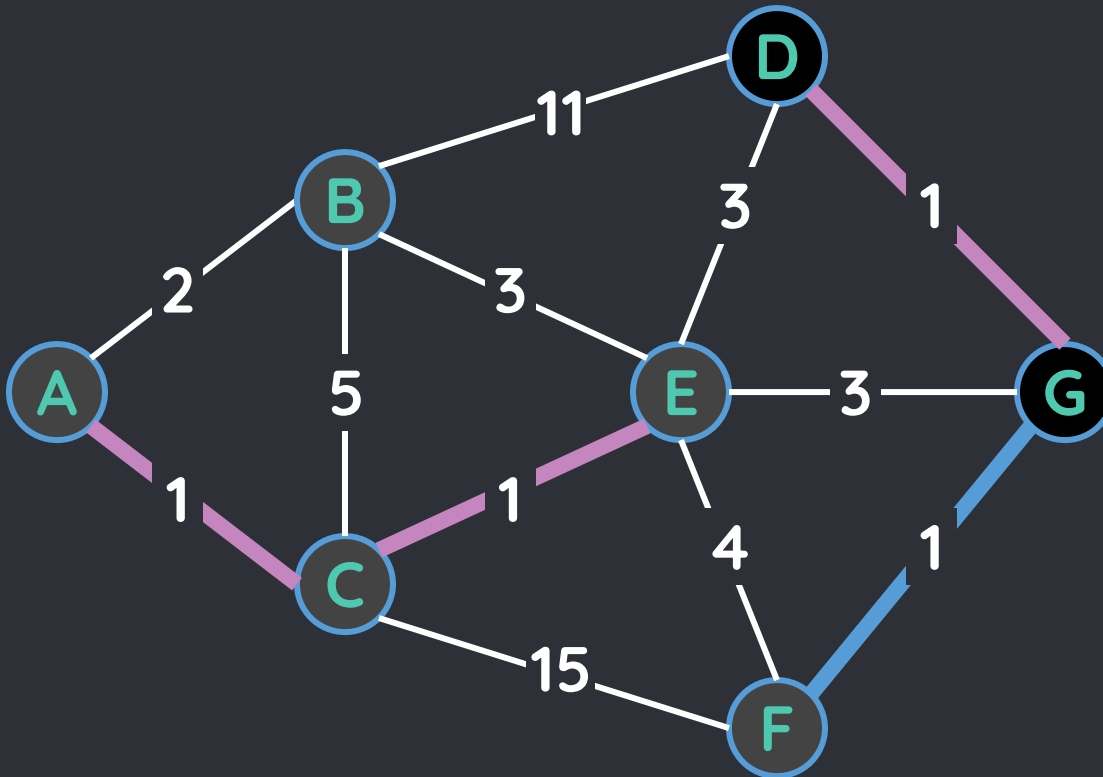


DS: [{A,C,E},{B},{D,G},{F}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

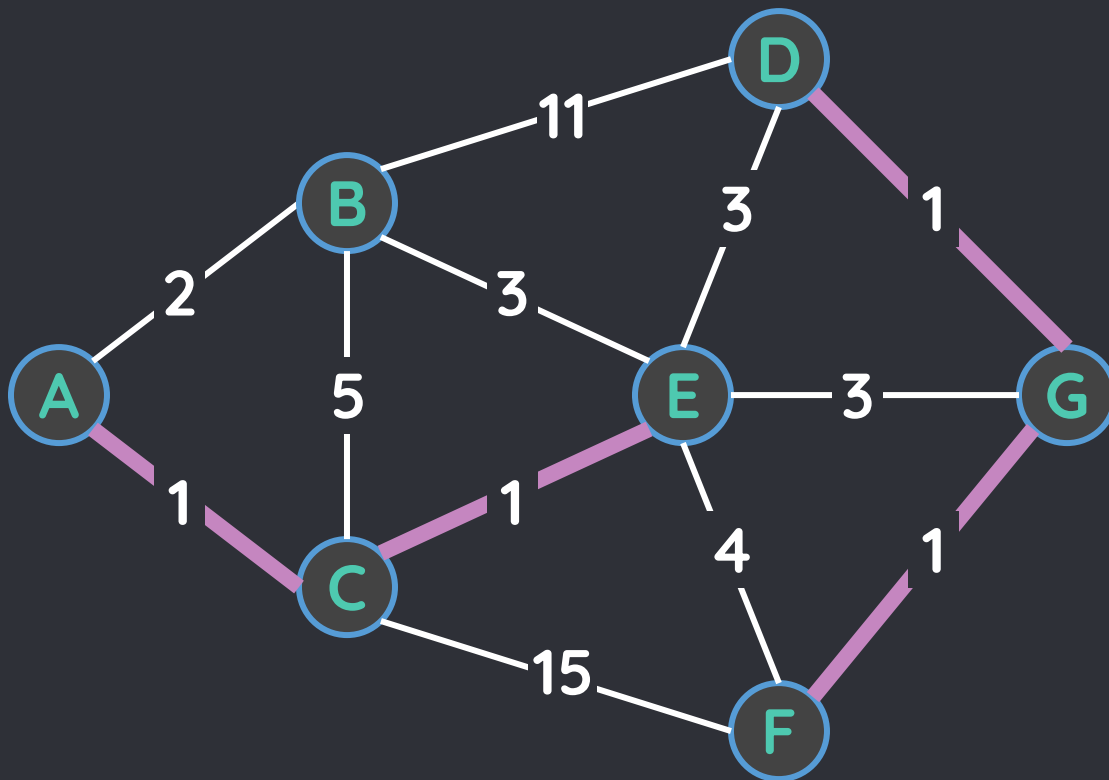
find(F,G) : False -> in MST



DS: [{A,C,E},{B},{D,G},{F}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

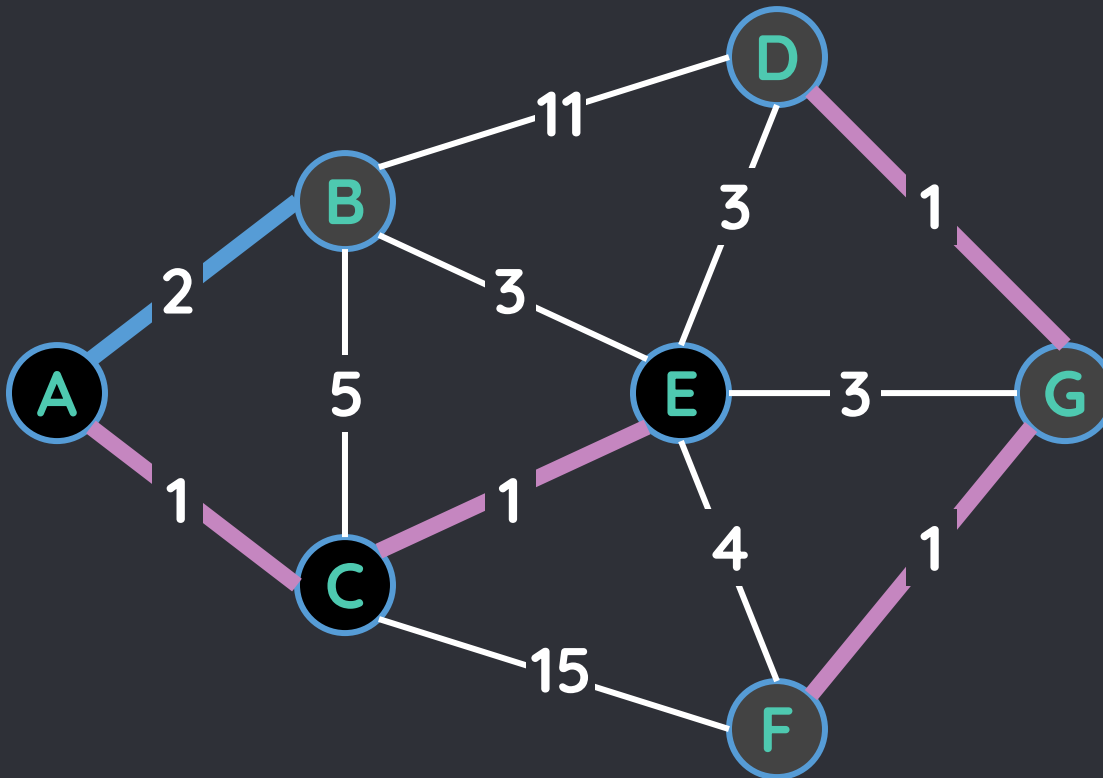


DS: [{A,C,E},{B},{D,F,G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

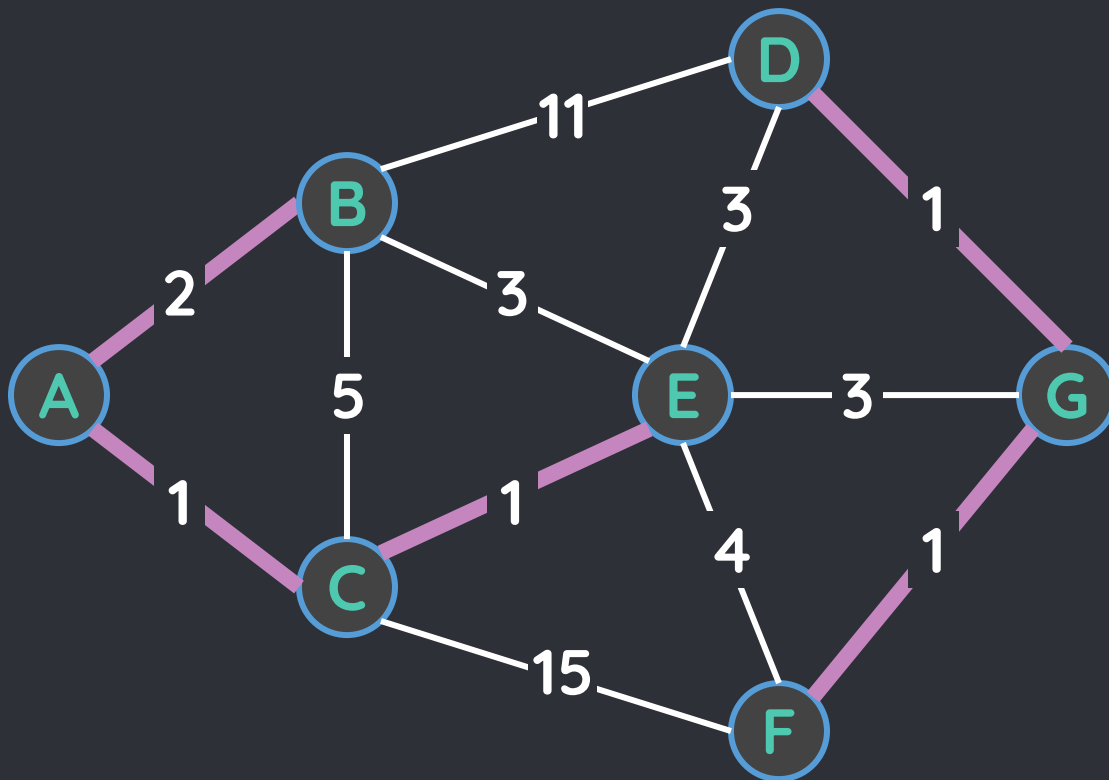
find(A,B) : False -> in MST



DS: [{A,C,E},{B},{D,F,G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

- Kruskal's Algorithm Optimized

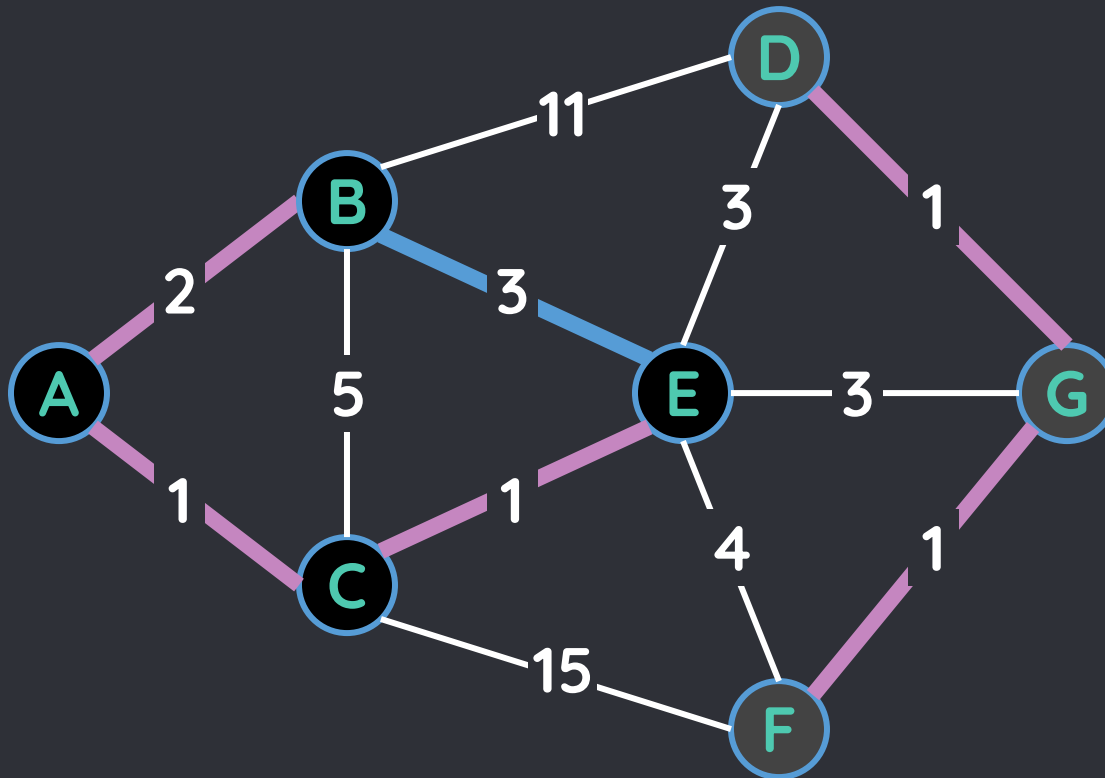


DS: [{A,B,C,E},{D,F,G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

find(B,E) : True -> not in MST

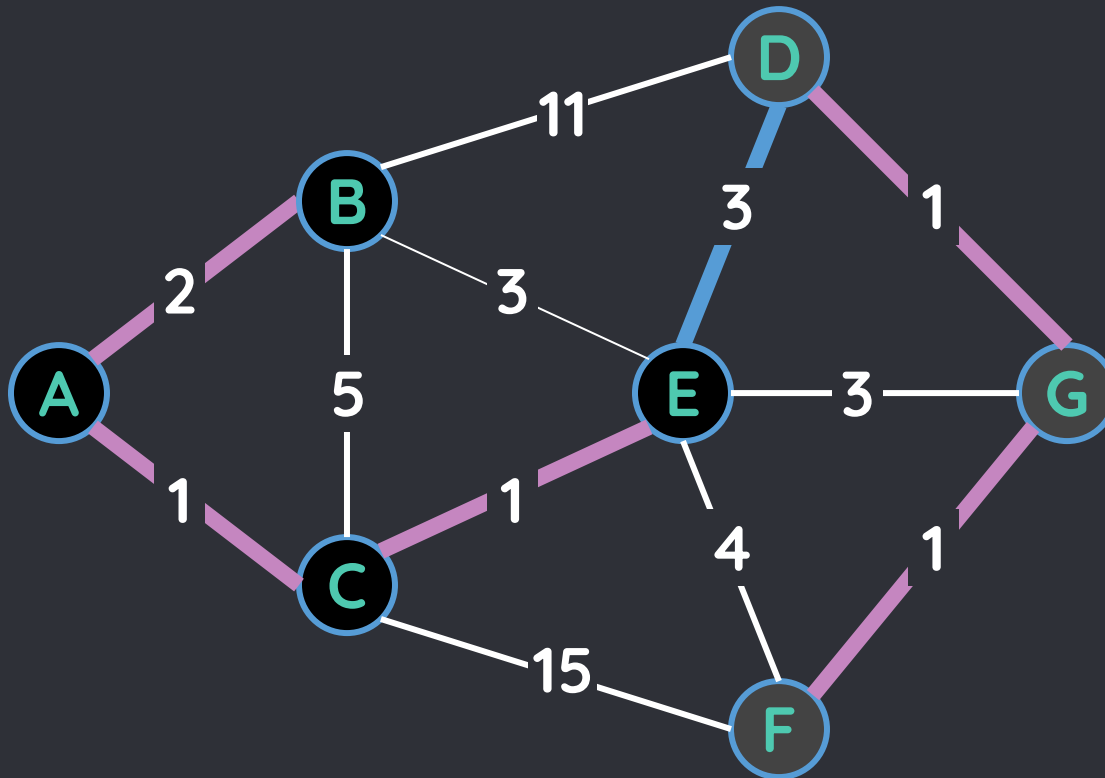


DS: [{A,B,C,E},{D,F,G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

find(D,E) : False -> in MST

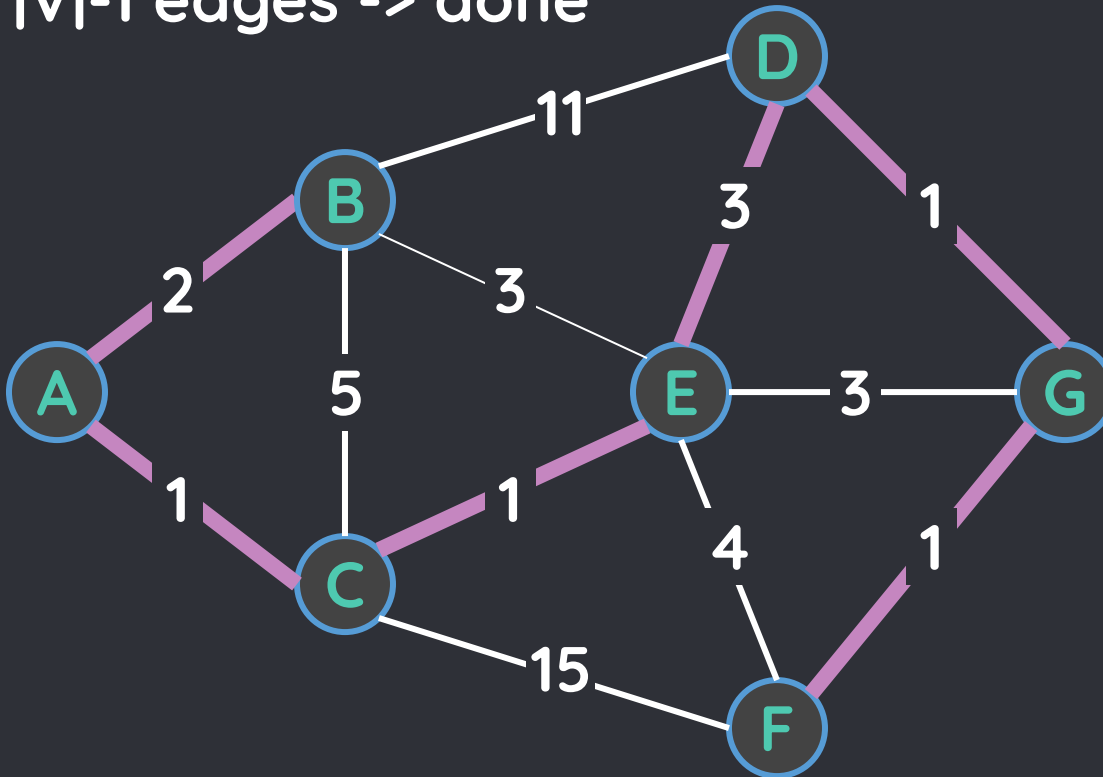


DS: [{A,B,C,E},{D,F,G}]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

• Kruskal's Algorithm Optimized

all nodes are connected,
 $|V|-1$ edges $\rightarrow$ done



DS: [**A,B,C,D,E,F,G**]

PQ	
value	key
(A,C)	1
(C,E)	1
(D,G)	1
(F,G)	1
(A,B)	2
(B,E)	3
(D,E)	3
(E,G)	3
(E,F)	4
(B,C)	5
(B,D)	11
(C,F)	15

Kruskal's Algorithm

Complexity

Space complexity: *worse than Prim*

- $O(|V| + |E|)$
 - $O(|E|)$ for edge priority queue
 - $O(|V|)$ for vertex disjoint sets

Time complexity:

- $O(|E| \log |E|)$ [unsorted edges]
 - $O(|E|)$ initialize heap
 - $O(|E| \log |E|)$ for heap extract
 - $O(|E| \alpha |V|)$ for find (WQUPC)
 - $O(|V| \alpha |V|)$ for union (WQUPC)
- $O((|V| + |E|) \alpha |V|)$ [presorted edges]

Kruskal's Algorithm

Complexity

Space complexity:

- $O(|V| + |E|)$
 - $O(|E|)$ for edge priority queue
 - $O(|V|)$ for vertex disjoint sets

$\alpha|V|$ is the inverse Ackermann function
which is almost practically constant
time

Time complexity:

- $O(|E| \log |E|)$ [unsorted edges]
 - $O(|E|)$ initialize heap
 - $O(|E| \log |E|)$ for heap extract
 - $O(|E| \alpha|V|)$ for find (WQUPC)
 - $O(|V| \alpha|V|)$ for union (WQUPC)
- $O((|V| + |E|) \alpha|V|)$ [presorted edges]

$\Rightarrow O(|E|)$
 $\Rightarrow O(|V|)$

$\hookrightarrow O(|V| + |E|)$ this is faster than Prim

• Prim's vs Kruskal's Algorithm

Algorithm	Space Complexity	Time Complexity
Prim's	$O(V)$	$O((V + E) \log V)$
Kruskal's	$O(V + E)$	$O(E \log E)$ - Unsorted $O((V + E) \alpha V)$ - Sorted

Summary:

- Prim's algorithm has a lower space and time complexity for graphs where $|E| \gg |V|$
- Kruskal's algorithm has the advantage of being practically linear time (instead of log-linear) if the edge list is already sorted based on weight

dense
sparse

• Prim's vs Kruskal's Algorithm

○ Summary:

- Prim's algorithm has a lower space and time complexity for graphs where $|E| \gg |V|$
- Kruskal's algorithm has the advantage of being practically linear time (instead of log-linear) if the edge list is already sorted based on weight

Thanks!

ANY QUESTIONS?

Post at our Piazza page